

Introduction to Software Testing

Chapter 5.1

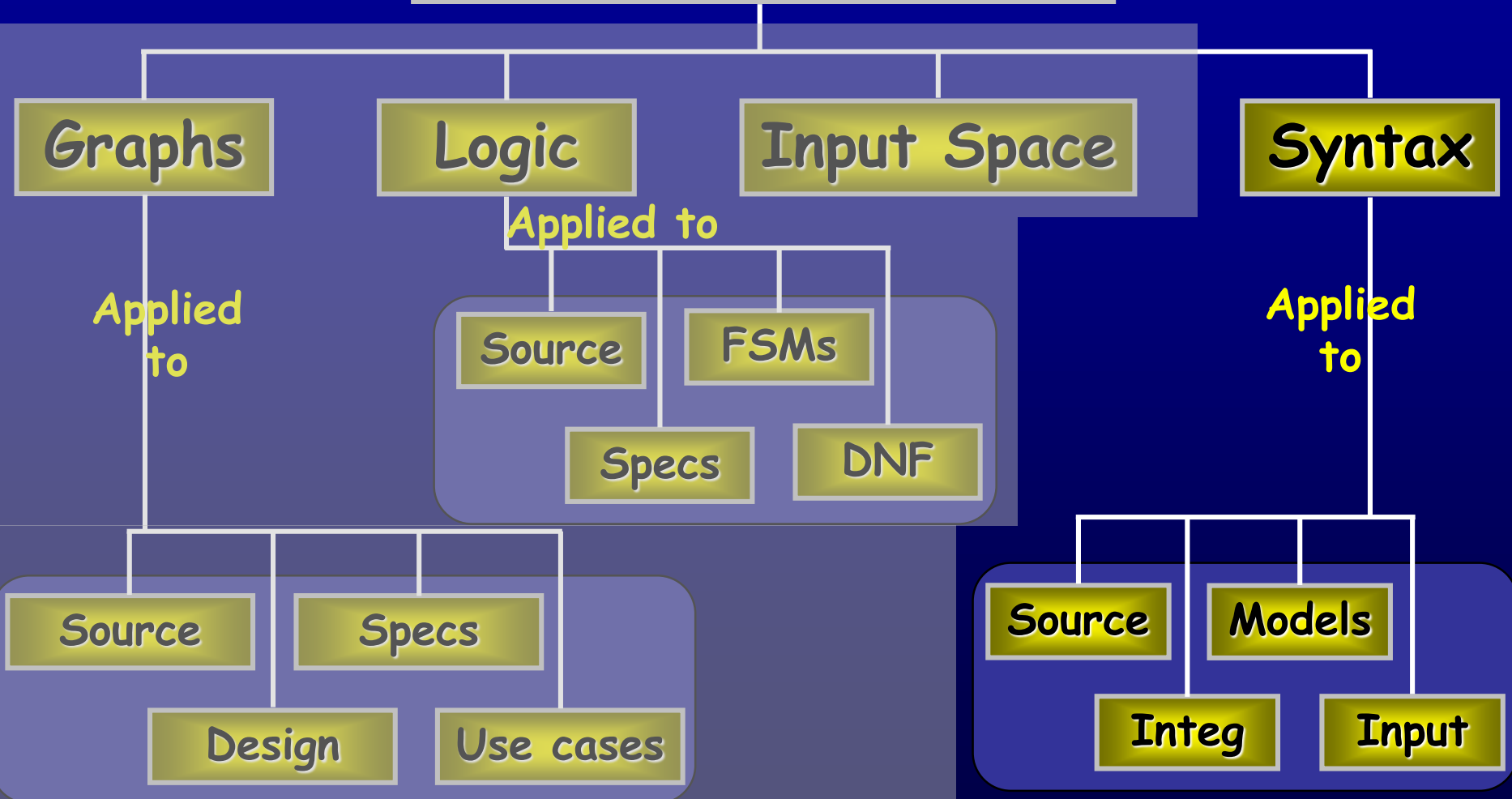
Syntax-based Testing

Paul Ammann & Jeff Offutt

<http://www.cs.gmu.edu/~offutt/softwaretest/>

Ch. 5 : Syntax Coverage

Four Structures for Modeling Software



Using the Syntax to Generate Tests (5.1)

- Lots of software artifacts follow **strict syntax** rules
- The syntax is often expressed as a **grammar** in a language such as BNF (Backus-Naur Form)
- **Syntactic descriptions** can come from many sources
 - Programs
 - Integration elements
 - Design documents
 - Input descriptions
- Tests are created with **two general goals**
 - **Cover** the syntax in some way
 - **Violate** the syntax (invalid tests)

Grammar Coverage Criteria

- Software engineering makes practical use of **automata theory** in several ways
 - **Programming languages** defined in BNF
 - **Program behavior** described as finite state
 - **Allowable inputs** defined by grammar
- A simple **regular expression**:

Make a fair example of context-free language

$(G s n \mid B t n)^*$

'*' is *closure operator*,
zero or more occurrences

'|' is *choice*, either
one can be used

- Any sequence of "**G s n**" and "**B t n**"
- '**G**' and '**B**' could be commands, methods, or events
- '**s**', '**t**', and '**n**' could represent arguments, parameters, or values
- '**s**', '**t**', and '**n**' could be literals or a set of values

Test Cases from Grammar

- A string that satisfies the derivation rules is said to be “*in the grammar*”
- A test case is a **sequence of strings** that satisfy the regular expression
- Suppose ‘s’, ‘t’ and ‘n’ are numbers

G 22 08 01 90

B 18 06 27 94

G 17 11 21 94

B 09 01 09 03

**Could be one test with four parts,
four separate tests, . . .**

BNF Grammars

Stream ::= action*

Start symbol

action ::= actG | actB

Non-terminals

actG ::= "G" s n

actB ::= "B" t n

Production rule

s ::= digit¹⁻³

t ::= digit¹⁻³

n ::= digit² "." digit² "." digit²

Terminals

digit ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" |
"7" | "8" | "9"

Using Grammars

Stream → action *

→ action action *

→ actG action*

→ G s n action*

→ G digit¹⁻³ digit² . digit² . digit² action*

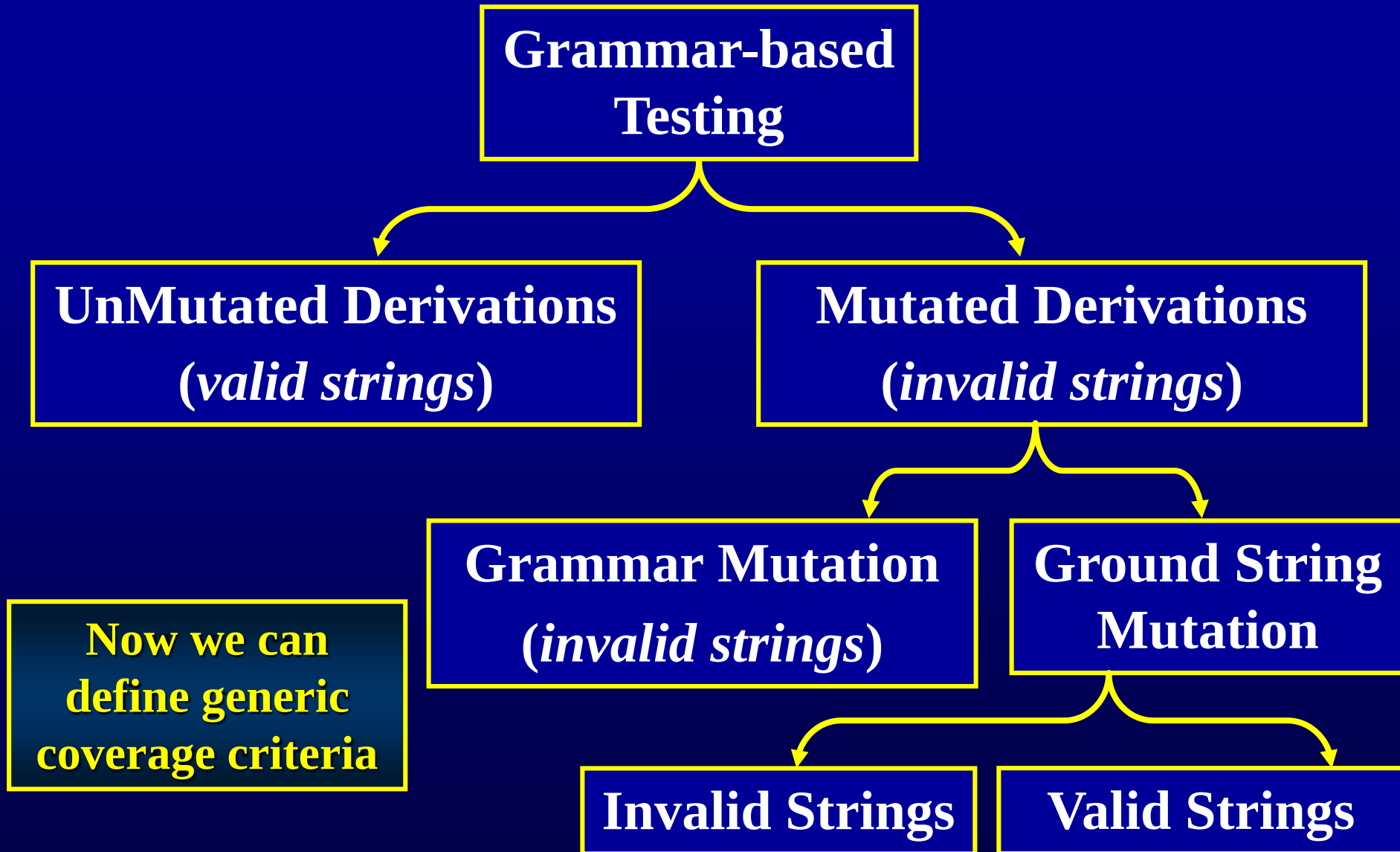
→ G digit digit digit digit.digit digit.digit digit action*

→ G 20 08.01.90 action*

...

- **Recognizer** : Given a string (or test), is the string in the grammar ?
 - This is called **parsing**
 - Tools exist to support **parsing**
 - Programs can use them for **input validation**
- **Generator** : Given a grammar, derive strings in the grammar

Mutation as Grammar-Based Testing



Syntax-based Coverage Criteria

- The most common and straightforward use every terminal and every production at least once

Terminal Symbol Coverage (TSC) : TR contains each terminal symbol t in the grammar G .

Production Coverage (PDC) : TR contains each production p in the grammar G .

- PDC subsumes TSC
- Grammars and graphs are interchangeable
 - PDC is equivalent to EC, TSC is equivalent to NC
- Other graph-based coverage criteria could be defined on grammar
 - But have not

Syntax-based Coverage Criteria

- A related criterion is the impractical one of deriving all possible strings

Derivation Coverage (DC) : TR contains every possible string that can be derived from the grammar G .

- The number of **TSC tests** is bound by the number of **terminal symbols**
 - 13 in the stream grammar
- The number of **PDC tests** is bound by the number of **productions**
 - 18 in the stream grammar
- The number of **DC tests** depends on the **details** of the grammar
 - **2,000,000,000** in the stream grammar for just one repetition!
- All TSC, PDC and DC tests are **in the grammar** ... how about tests that are **NOT in the grammar** ?

Mutation Testing

- Grammars describe both **valid** and **invalid** strings
- Both types can be produced as **mutants**
- A mutant is a **variation** of a valid string
 - Mutants may be valid or invalid strings
- Mutation is based on “**mutation operators**” and “**ground strings**”

What is Mutation ?

General View

We are performing mutation analysis whenever we

- use well defined **rules**
- defined on **syntactic descriptions**
- to make **systematic changes**
- to the **syntax** or to **objects** developed from the syntax

mutation
operators

grammars

Applied universally or
according to empirically
verified distributions

grammar

ground
strings

Mutation Testing

- **Ground string:** A **string** in the grammar
 - The term “ground” is used as a reference to algebraic ground terms
- **Mutation Operator :** A rule that specifies **syntactic variations** of strings generated from a grammar
- **Mutant :** The result of **one application** of a mutation operator
 - A mutant is a string

Mutants and Ground Strings

- The key to mutation testing is the **design** of the mutation operators
 - Well designed **operators** lead to powerful testing
- Sometimes **mutant strings** are based on ground strings
- Sometimes they are derived directly **from the grammar**
 - **Ground** strings are used for **valid** tests
 - **Invalid** tests do not need ground strings

Valid Mutants

Ground Strings

G 22 08.01.90

B 18 06.27.94

Mutants

B 22 08.01.90

B 45 06.27.94

Invalid Mutants

7 22 08.01.90

B 18 06.27.1

Questions About Mutation

- Should **more than one operator** be applied at the same time ?
 - Or, should a mutated string contain one mutated element or several?
 - Usually not – multiple mutations can interfere with each other
 - Extensive experience with program-based mutation indicates not
 - Recent research is finding exceptions
- Should **every possible application** of a mutation operator be considered ?
 - Necessary with program-based mutation
- Mutation operators were defined for several **languages**
 - Several programming languages (*Fortran, Lisp, Ada, C, C++, Java*)
 - Specification languages (*SMV, Z, Object-Z, algebraic specs*)
 - Modeling languages (*Statecharts, activity diagrams*)
 - Input grammars (*XML, SQL, HTML*)

2013/05/14 stopped here.

Killing Mutants

- When ground strings are mutated to create valid strings, the hope is to exhibit **different behavior** from the ground string
- This is normally used when the grammars are **programming languages**, the strings are **programs**, and the ground strings are **pre-existing** programs
- **Killing Mutants** : Given a mutant $m \in M$ for a derivation D and a test t , t is said to kill m if and only if the output of t on D is different from the output of t on m
- The derivation D may be represented by the list of productions or by the final string

Syntax-based Coverage Criteria

- Coverage is defined in terms of killing mutants

Mutation Coverage (MC) : For each $m \in M$, TR contains exactly one requirement, to kill m .

- Coverage in mutation equates to number of mutants killed
- The amount of mutants killed is called the **mutation score**

Syntax-based Coverage Criteria

- When creating invalid strings, we just apply the operators
- This results in two simple criteria
- It makes sense to either use every operator once or every production once

Mutation Operator Coverage (MOC) : For each mutation operator, TR contains exactly one requirement, to create a mutated string m that is derived using the mutation operator.

Mutation Production Coverage (MPC) : For each mutation operator, TR contains several requirements, to create one mutated string m that includes every production that can be mutated by that operator.

Example

Stream ::= action*	Grammar
action ::= actG actB	
actG ::= "G" s n	
actB ::= "B" t n	
s ::= digit¹⁻³	
t ::= digit¹⁻³	
n ::= digit² "." digit² "." digit²	
digit ::= "0" "1" "2" "3" "4" "5" "6" "7" "8" "9"	

Ground String

G 22 08.01.90

B 18 06.27.94

Mutation Operators

- *Exchange actG and actB*
- *Replace digits with all other digits*

Mutants using MOC

B 22 08.01.90

B 19 06.27.94

Mutants using MPC

B 22 08.01.90 G 18 06.27.94

G 12 08.01.90 B 11 06.27.94

G 32 08.01.90 B 12 06.27.94

G 42 08.01.90 B 13 06.27.94

G 52 08.01.90 B 14 06.27.94

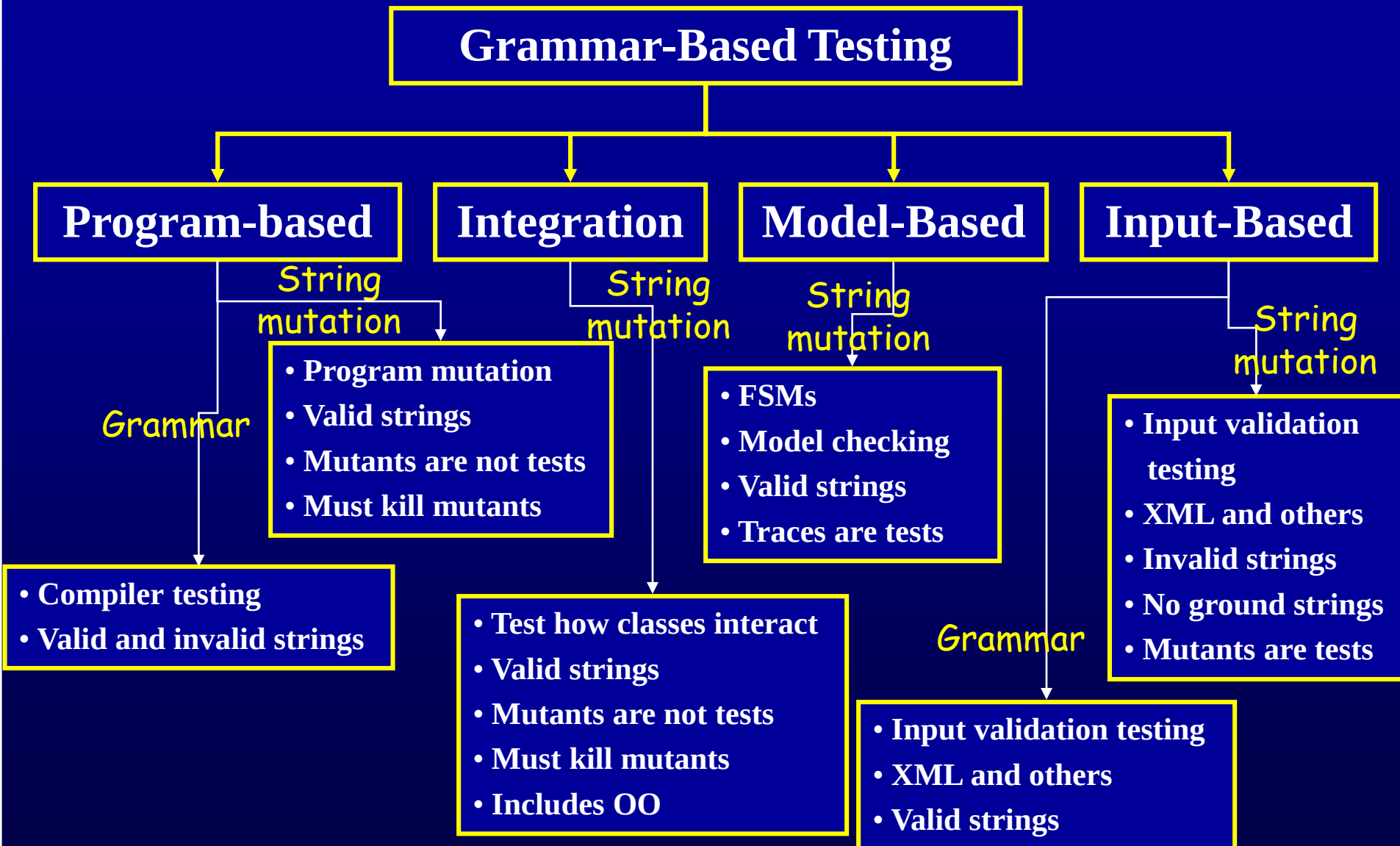
...

...

Mutation Testing

- The **number of test requirements** for mutation depends on two things
 - The **syntax** of the artifact being mutated
 - The mutation **operators**
- Mutation testing is very difficult to apply **by hand**
- Mutation testing is very effective – considered the “**gold standard**” of testing
- Mutation testing is often used to **evaluate** other criteria

Instantiating Grammar-Based Testing



Structure of Chapter

	Program-based	Integration	Model-based	Input space
Grammar	5.2.1	5.3.1	5.4.1	5.5.1
Grammar	Programming languages	No known applications	Algebraic specifications	Input languages, including XML
Summary	Compiler testing			Input space testing
Valid?	Valid & invalid			Valid
Mutation	5.2.2	5.3.2	5.4.2	5.5.2
Grammar	Programming languages	Programming languages	FSMs	Input languages, including XML
Summary	Mutates programs	Tests integration	Model checking	Error checking
Ground?	Yes	Yes	Yes	No
Valid?	Yes, must compile	Yes, must compile	Yes	No
Tests?	Mutants not tests	Mutants not tests	Traces are tests	Mutants are tests
Killing	Yes	Yes	Yes	No
Notes	Strong and weak. Subsumes other techniques	Includes OO testing		Sometimes the grammar is mutated

Introduction to Software Testing

Chapter 5.2

Program-based Grammars

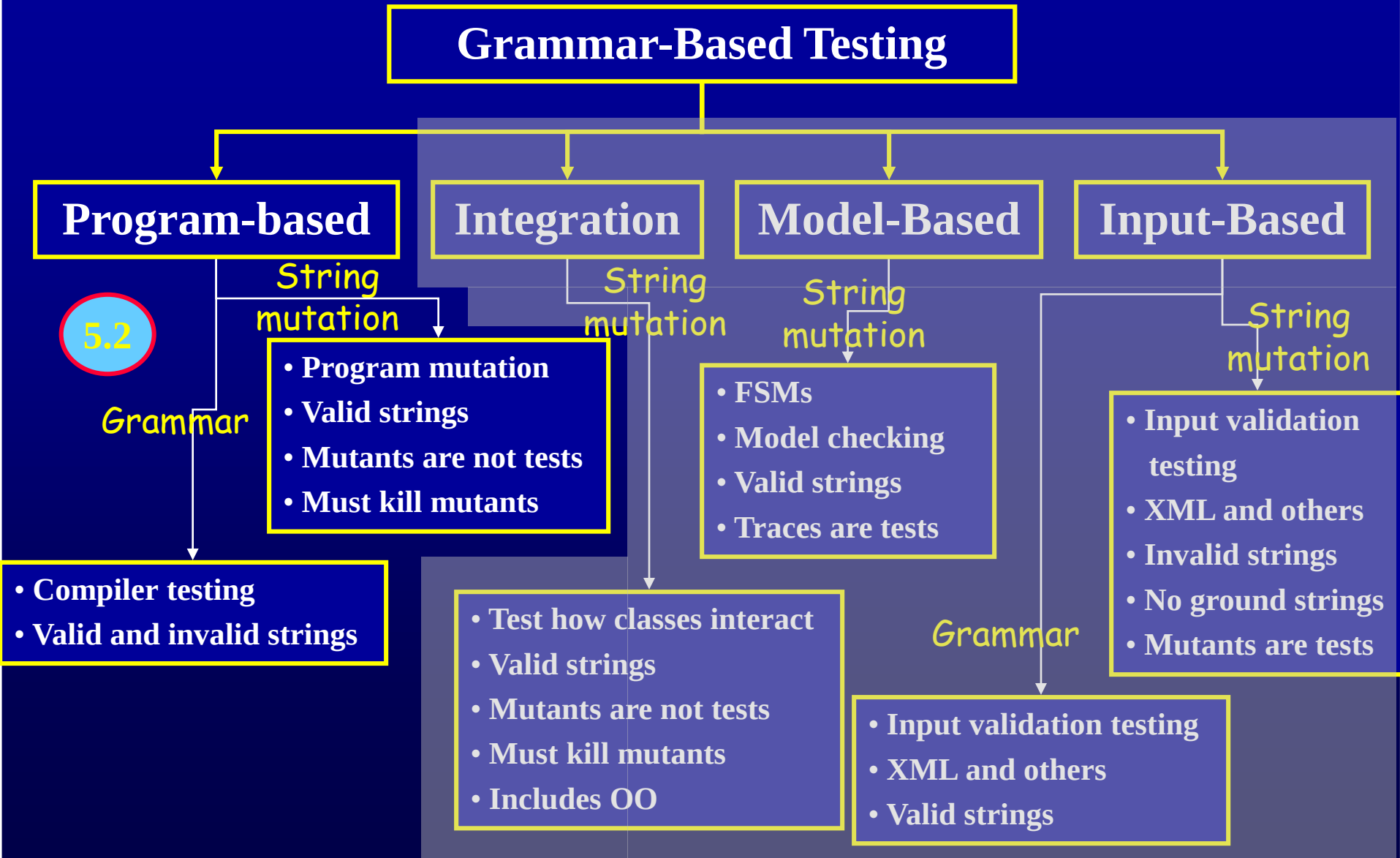
Paul Ammann & Jeff Offutt

<http://www.cs.gmu.edu/~offutt/softwaretest/>

Applying Syntax-based Testing to Programs

- Syntax-based criteria originated with programs and have been used most with programs
- BNF criteria are most commonly used to test compilers
- Mutation testing criteria are most commonly used for unit testing and integration testing of classes

Instantiating Grammar-Based Testing



BNF Testing for Compilers (5.2.1)

- **Testing compilers is very complicated**
 - Millions of correct programs !
 - Compilers must recognize and reject incorrect programs
- **BNF criteria can be used to generate programs to test all language features that compilers must process**
- **This is a very specialized application and not discussed in detail**

Program-based Grammars (5.2.2)

- The original and most widely known application of syntax-based testing is to modify programs
- Operators modify a ground string (program under test) to create mutant programs
- Mutant programs must compile correctly (valid strings)
- Mutants are not tests, but used to find tests
- Once mutants are defined, tests must be found to cause mutants to fail when executed
- This is called “killing mutants”

Killing Mutants

Given a mutant $m \in M$ for a ground string program P and a test t , t is said to **kill** m if and only if the output of t on P is different from the output of t on m .

- If mutation operators are designed well, the resulting tests will be very powerful
- Different operators must be defined for different programming languages and goals
- Testers can keep adding tests until all mutants have been killed
 - **Dead mutant** : A test case has killed it
 - **Stillborn mutant** : Syntactically illegal
 - **Trivial mutant** : Almost every test can kill it
 - **Equivalent mutant** : No test can kill it (equivalent to original program)

Program-based Grammars

Original Method

```
int Min (int A, int B)
{
    int minVal;
    minVal = A;
    if (B < A)
    {
        minVal = B;
    }
    return (minVal);
} // end Min
```

6 mutants

**Each represents a
separate program**

With Embedded Mutants

```
int Min (int A, int B)
{
    int minVal;
    minVal = A;
    Δ 1 minVal = B;
    if (B < A)
    Δ 2 if (B > A)
    Δ 3 if (B < minVal)
    {
        minVal = B;
    Δ 4 Bomb ();
    Δ 5 minVal = A;
    Δ 6 minVal = failOnZero (B);
    }
    return (minVal);
} // end Min
```

*Replace one variable
with another*

Changes operator

*Immediate runtime
failure ... if reached*

*Immediate runtime
failure if B==0 else
does nothing*

Syntax-Based Coverage Criteria

Mutation Coverage (MC) : For each $m \in M$, TR contains exactly one requirement, to kill m .

- The RIP model from chapter 1:
 - **Reachability** : The test causes the **faulty statement** to be reached (in mutation – the **mutated** statement)
 - **Infection** : The test causes the faulty statement to result in an **incorrect state**
 - **Propagation** : The incorrect state **propagates** to incorrect output
- The RIP model leads to **two variants** of mutation coverage ...

Syntax-Based Coverage Criteria

1) Strongly Killing Mutants:

Given a mutant $m \in M$ for a program P and a test t , t is said to **strongly kill** m if and only if the **output** of t on P is different from the output of t on m

2) Weakly Killing Mutants:

Given a mutant $m \in M$ that modifies a location l in a program P , and a test t , t is said to **weakly kill** m if and only if the **state** of the execution of P on t is different from the state of the execution of m immediately on t after l

- Weakly killing satisfies **reachability** and **infection**, but not **propagation**

Weak Mutation

Weak Mutation Coverage (WMC) : For each $m \in M$, TR contains exactly one requirement, to weakly kill m .

- “Weak mutation” is so named because it is **easier to kill** mutants under this assumption
- Weak mutation also requires **less analysis**
- Some mutants can be killed under weak mutation but not under strong mutation (**no propagation**)
- In practice, there is **little difference**

Weak Mutation Example

- Mutant 1 in the Min() example is:

```
minVal = A;  
Δ 1 minVal = B;  
if (B < A)  
    minVal = B;
```

- The complete test specification to kill mutant 1:
- Reachability : *true* // Always get to that statement
- Infection : $A \neq B$
- Propagation: $(B < A) = \text{false}$ // Skip the next assignment
- Full Test Specification : $\text{true} \wedge (A \neq B) \wedge ((B < A) = \text{false})$
 $\equiv (A \neq B) \wedge (B \geq A)$
 $\equiv (B > A)$
- **(A = 5, B = 3)** will weakly kill mutant 1, but not strongly

Equivalent Mutation Example

- Mutant 3 in the Min() example is equivalent:

```
minVal = A;  
if (B < A)  
  Δ 3 if (B < minVal)
```

- The infection condition is “**(B < A) != (B < minVal)**”
- However, the previous statement was “**minVal = A**”
 - Substituting, we get: “**(B < A) != (B < A)**”
- Thus no input can kill this mutant

Strong Versus Weak Mutation

```
1  boolean isEven (int X)
2  {
3      if (X < 0)
4          X = 0 - X;
5      if (double) (X/2) == ((double) X) / 2.0
6          return (true);
7      else
8          return (false);
9  }
```

Δ 4

X = 0;

Reachability : $X < 0$

Infection : $X \neq 0$

(X = -6) will kill mutant
4 under weak mutation

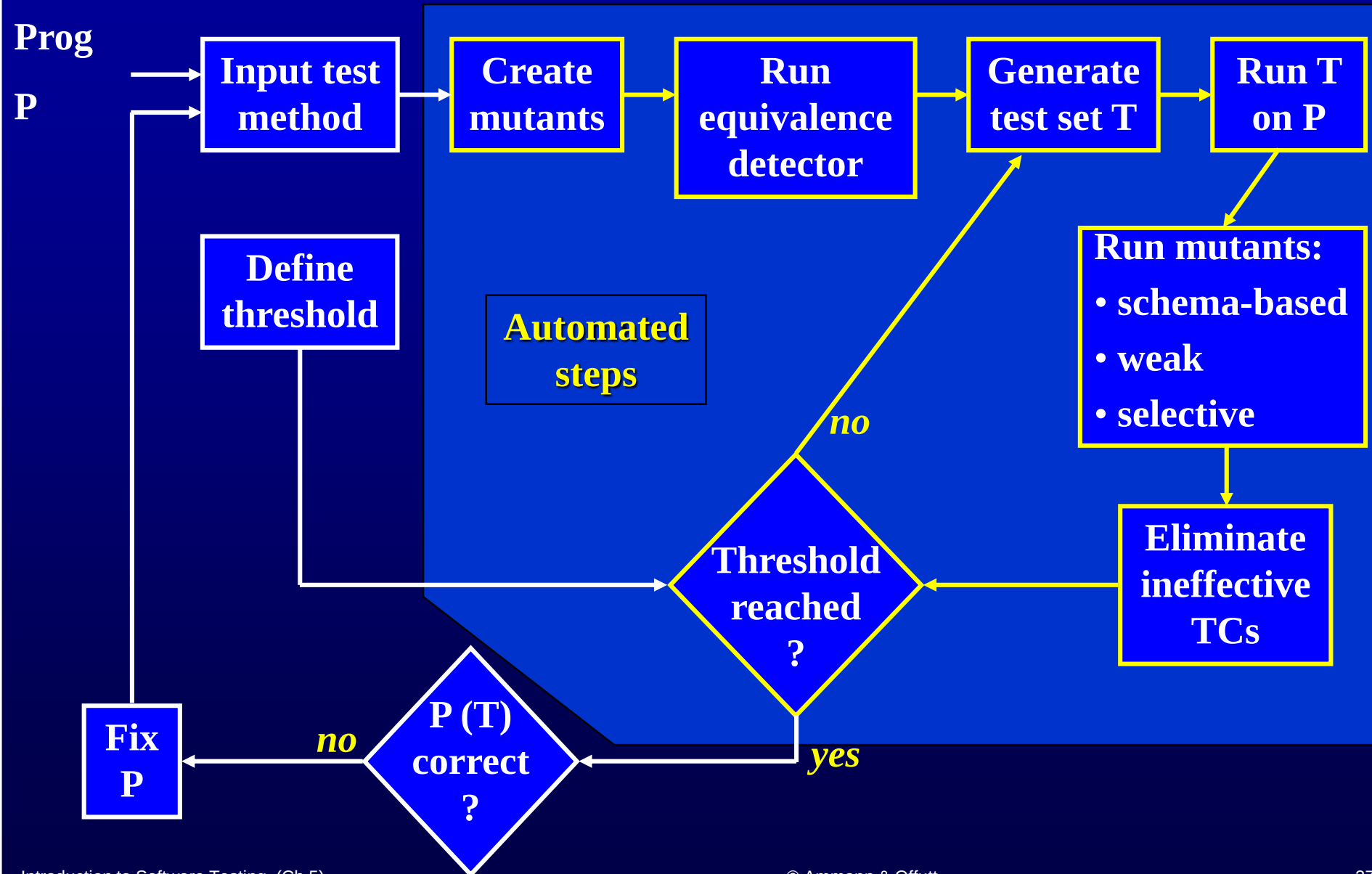
Propagation :

$((\text{double}) ((0-X)/2) == ((\text{double}) 0-X) / 2.0)$
 $\neq ((\text{double}) (0/2) == ((\text{double}) 0) / 2.0)$

That is, X is not even ...

Thus (X = -6) does not kill the mutant under
strong mutation

Testing Programs with Mutation



Why Mutation Works

Fundamental Premise of Mutation Testing

If the software contains a fault, there will usually be a set of mutants that can only be killed by a test case that also detects that fault

- This is not an absolute !
- The mutants guide the tester to a very effective set of tests
- A very challenging problem :
 - Find a **fault** and a set of **mutation-adequate tests** that do **not** find the fault
- Of course, this depends on the mutation operators ...

Designing Mutation Operators

- At the method level, mutation operators for different programming languages are similar
- Mutation operators do one of two things:
 - Mimic typical programmer mistakes (e.g., incorrect variable name)
 - Encourage common test heuristics (e.g., cause expressions to be 0)
- Researchers design lots of operators, then experimentally select the most useful

Effective Mutation Operators

If tests that are created specifically to kill mutants created by a collection of mutation operators $O = \{o1, o2, \dots\}$, they also kill mutants created by all remaining mutation operators with very high probability, then O defines an *effective* set of mutation operators

Mutation Operators for Java

1. ABS — *Absolute Value Insertion*:

Each arithmetic expression (and subexpression) is modified by the functions *abs()*, *negAbs()*, and *failOnZero()*.

2. AOR — *Arithmetic Operator Replacement*:

Each occurrence of one of the arithmetic operators $+$, $-$, $*$, $/$, and $\%$ is replaced by each of the other operators. In addition, each is replaced by the special mutation operators *leftOp*, and *rightOp*.

3. ROR — *Relational Operator Replacement*:

Each occurrence of one of the relational operators ($<$, $\leq$, $>$, $\geq$, $=$, $\neq$) is replaced by each of the other operators and by *falseOp* and *trueOp*.

Mutation Operators for Java (2)

4. *COR* — *Conditional Operator Replacement*:

Each occurrence of one of the logical operators (and - &&, or - || , and with no conditional evaluation - &, or with no conditional evaluation - |, not equivalent - ^) is replaced by each of the other operators; in addition, each is replaced by falseOp, trueOp, leftOp, and rightOp.

5. *SOR* — *Shift Operator Replacement*:

Each occurrence of one of the shift operators <<, >>, and >>> is replaced by each of the other operators. In addition, each is replaced by the special mutation operator leftOp.

6. *LOR* — *Logical Operator Replacement*:

Each occurrence of one of the logical operators (bitwise and - &, bitwise or - |, exclusive or - ^) is replaced by each of the other operators; in addition, each is replaced by leftOp and rightOp.

Mutation Operators for Java (3)

7. *ASR — Assignment Operator Replacement:*

Each occurrence of one of the assignment operators (`+=`, `-=`, `*=`, `/=`, `%=`, `&=`, `|=`, `^=`, `<<=`, `>>=`, `>>>=`) is replaced by each of the other operators.

8. *UOI — Unary Operator Insertion:*

Each unary operator (arithmetic `+`, arithmetic `-`, conditional `!`, logical `~`) is inserted in front of each expression of the correct type.

9. *UOD — Unary Operator Deletion:*

Each unary operator (arithmetic `+`, arithmetic `-`, conditional `!`, logical `~`) is deleted.

Mutation Operators for Java (4)

10. SVR — Scalar Variable Replacement:

Each variable reference is replaced by every other variable of the appropriate type that is declared in the current scope.

11. BSR — Bomb Statement Replacement:

Each statement is replaced by a special Bomb() function.

Summary : Subsumption of Other Criteria

- Mutation is widely considered the **strongest** test criterion
 - And most **expensive** !
 - By far the most test requirements (each mutant)
 - Not always the most tests
- Mutation **subsumes** other criteria by including specific mutation operators
- Subsumption actually only makes sense for **weak mutation** – other criteria impose local requirements, like weak mutation
 - Node coverage
 - Edge coverage
 - Clause coverage
 - General active clause coverage: **Yes – Requirement on single tests**
 - Correlated active clause coverage: **No – Requirement on *pairs* of tests**
 - All-defs data flow coverage

Introduction to Software Testing

Chapter 5.3

Integration and Object-Oriented Testing

Paul Ammann & Jeff Offutt

<http://www.cs.gmu.edu/~offutt/softwaretest/>

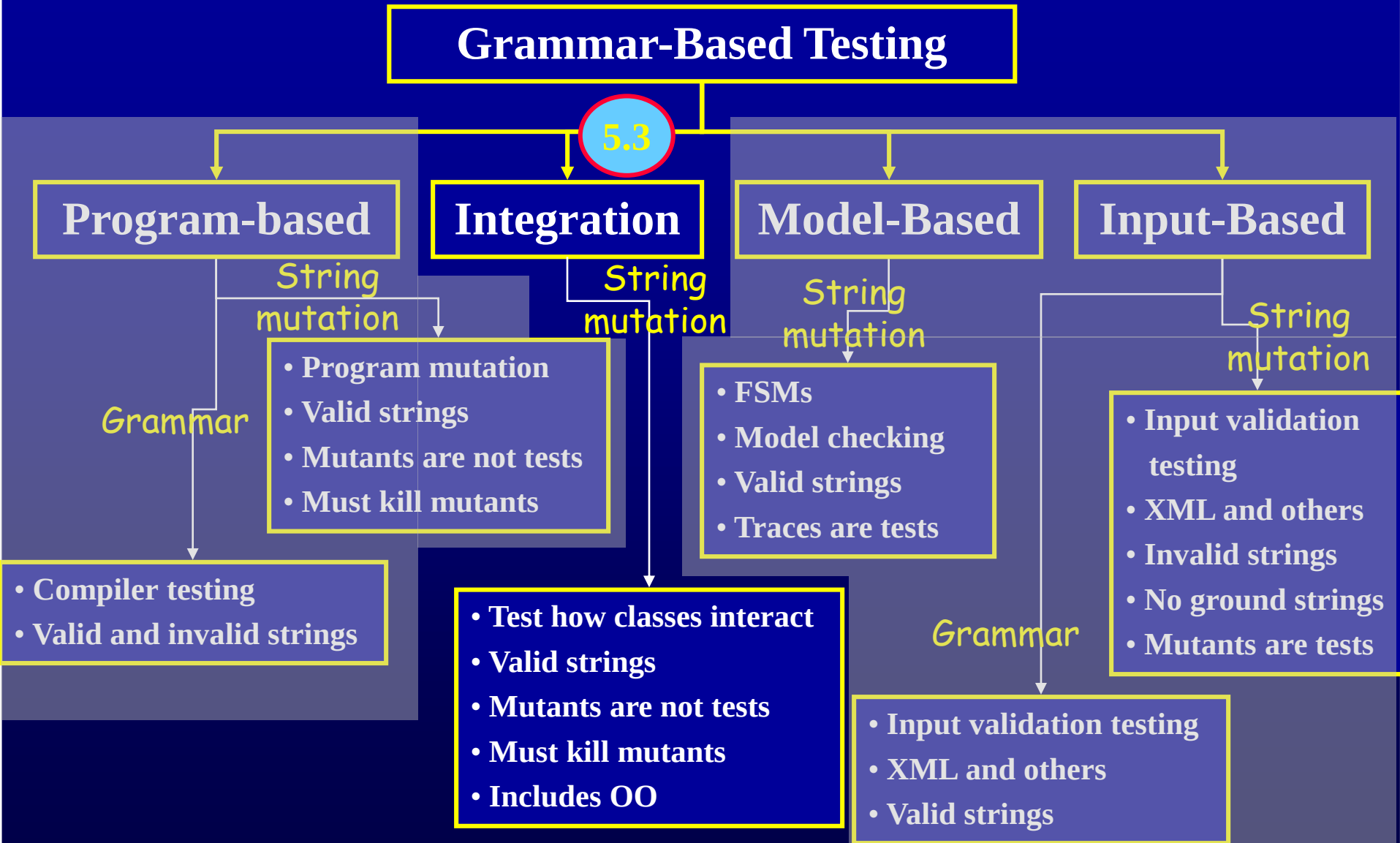
Integration and Object-Oriented Testing

Integration Testing

Testing connections among separate program units

- In Java, testing the way **classes**, **packages** and **components** are connected
 - “*Component*” is used as a generic term
- This tests **features** that are unique to object-oriented programming languages
 - inheritance, polymorphism and dynamic binding
- Integration testing is often based on **couplings** – the explicit and implicit relationships among software components

Instantiating Grammar-Based Testing



Grammar Integration Testing (5.3.1)

There is no known use of grammar testing at the integration level

Integration Mutation (5.3.2)

- Faults related to component integration often depend on a **mismatch of assumptions**
 - Callee thought a list was sorted, caller did not
 - Callee thought all fields were initialized, caller only initialized some of the fields
 - Caller sent values in kilometers, callee thought they were miles
- Integration mutation focuses on **mutating the connections** between components
 - Sometimes called “*interface mutation*”
 - Both caller and callee methods are considered

Four Types of Mutation Operators

- Change a **calling** method by **modifying values that are sent** to a called method
- Change a **calling** method by **modifying the call**
- Change a **called** method by **modifying values that enter and leave** a method
 - Includes parameters as well as variables from higher scopes (class level, package, public, etc.)
- Change a **called** method by **modifying return statements** from the method

Five Integration Mutation Operators

1. *IPVR — Integration Parameter Variable Replacement*

Each parameter in a method call is replaced by each other variable in the scope of the method call that is of compatible type.

- **This operator replaces primitive type variables as well as objects.**

2. *IUOI — Integration Unary Operator Insertion*

Each expression in a method call is modified by inserting all possible unary operators in front and behind it.

- **The unary operators vary by language and type**

3. *IPEX — Integration Parameter Exchange*

Each parameter in a method call is exchanged with each parameter of compatible types in that method call.

- **max (a, b) is mutated to max (b, a)**

Five Integration Mutation Operators (2)

4. *IMCD* — *Integration Method Call Deletion*

Each method call is deleted. If the method returns a value and it is used in an expression, the method call is replaced with an appropriate constant value.

- **Method calls that return objects are replaced with calls to “new ()”**

5. *IREM* — *Integration Return Expression Modification*

Each expression in each return statement in a method is modified by applying the UOI and AOR operators.

Integration Mutation Operators—Example

1. IPVR – *Integration Parameter Variable Replacement*

```
MyObject a, b;  
    . . .  
    callMethod (a);  
Δ callMethod (b);
```

2. IUOI – *Integration Unary Operator Insertion*

```
callMethod (a);  
Δ callMethod (a++);
```

Integration Mutation Operators—Example

3. IPEX – *Integration Parameter Exchange*

```
Max (a, b);  
Δ Max (b, a);
```

4. IMCD – *Integration Method Call Deletion*

```
X = Max (a, b);  
Δ X = new Integer (0);
```

5. IREM – *Integration Return Expression Modification*

```
int myMethod ()  
{  
    return a;  
Δ   return ++a;  
}
```

Object-Oriented Mutation

Testing Levels

intra-method

inter-method

intra-class

inter-class

integration mutation operators

- These five operators can be applied to **non-OO** languages
 - C, Pascal, Ada, Fortran, ...
- They do **not support** object oriented features
 - **Inheritance, polymorphism, dynamic binding**
- Two other language features that are often lumped with OO features are **information hiding (encapsulation)** and **overloading**
- Even experienced programmers often get encapsulation and access control wrong

**The following five pages
to be presentated by 5 groups**

Encapsulation, Information Hiding and Access Control

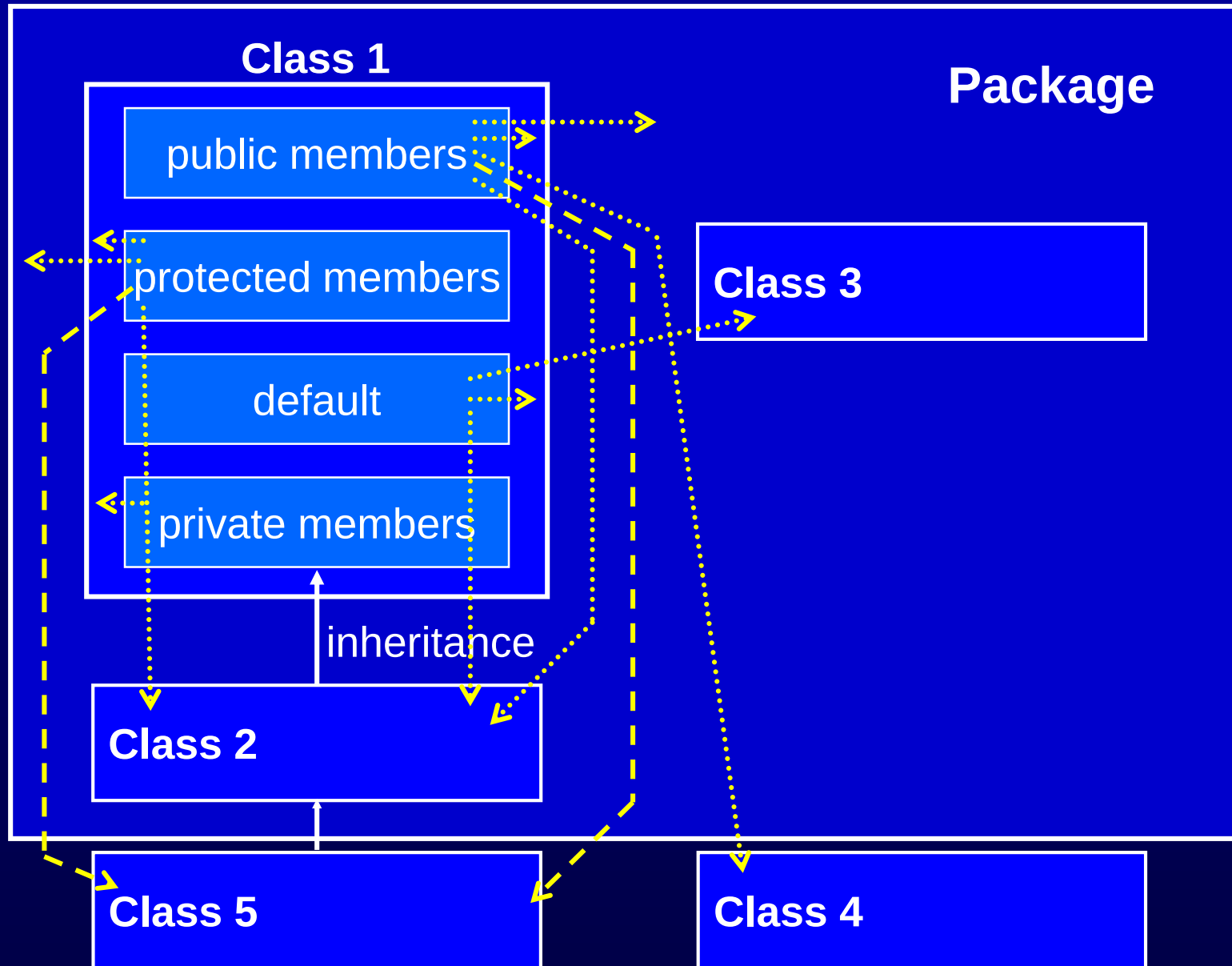
- **Encapsulation** : An abstraction mechanism to implement information hiding, which is a design technique that attempts to protect parts of the design from parts of the implementation
 - Objects can restrict access to their member variables and methods
- Java provides four **access levels** (C++ & C# are similar)
 - private
 - protected
 - public
 - default (also called package)
- Often **not used correctly** or understood, especially for programmers who are not well educated in design

Access Control in Java

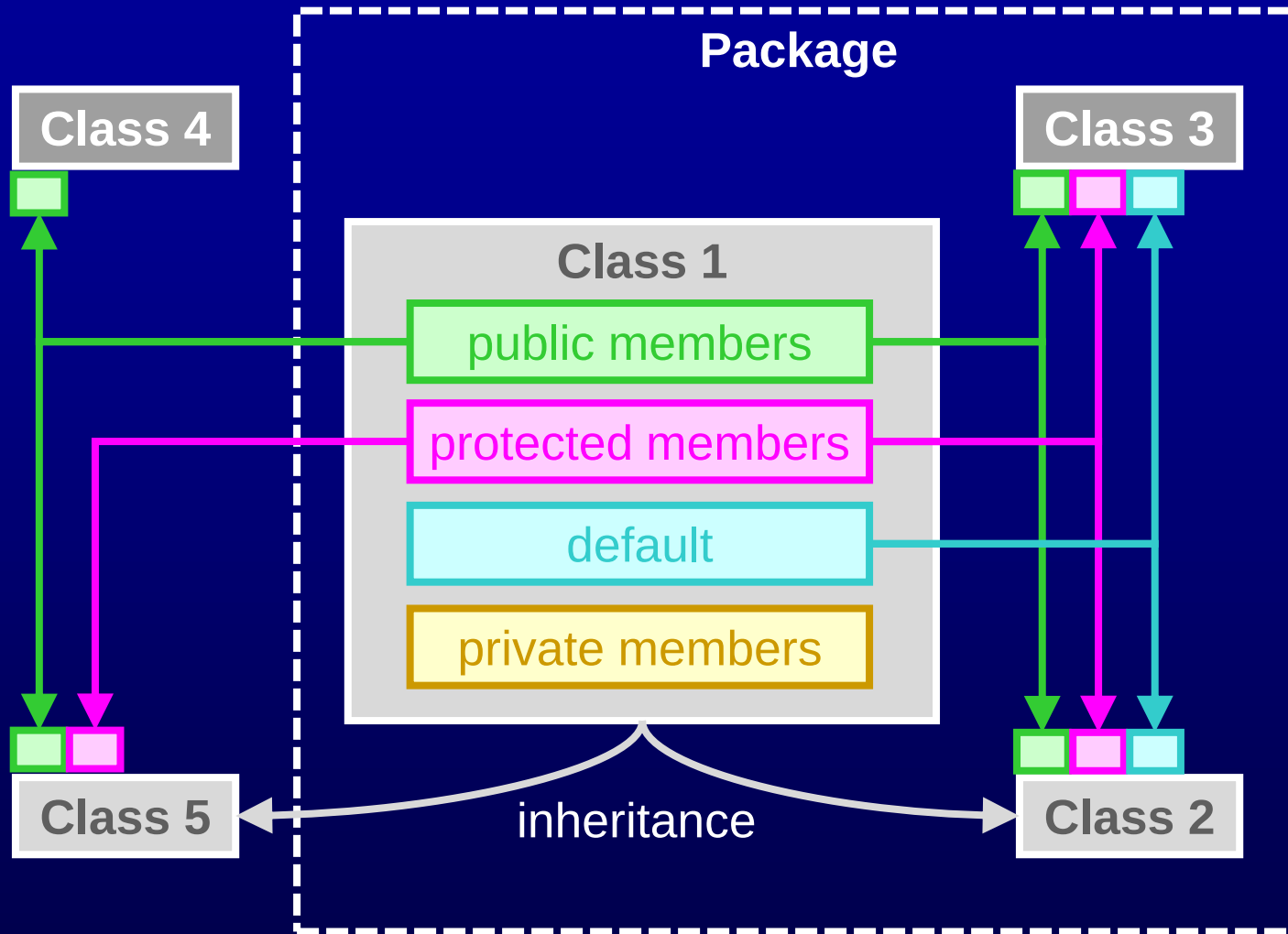
Specifier	Same class	Same package	Different package subclass	Different package non-subclass
private	Y	n	n	n
package	Y	Y	n	n
protected	Y	Y	Y	n
public	Y	Y	Y	Y

- Most class variables should be **private**
- **Public** variables should seldom be used
- **Protected** variables are particularly **dangerous** – future programmers can accidentally override (by using the same name) or accidentally use (by mis-typing a similar name)
 - They should be called “unprotected”

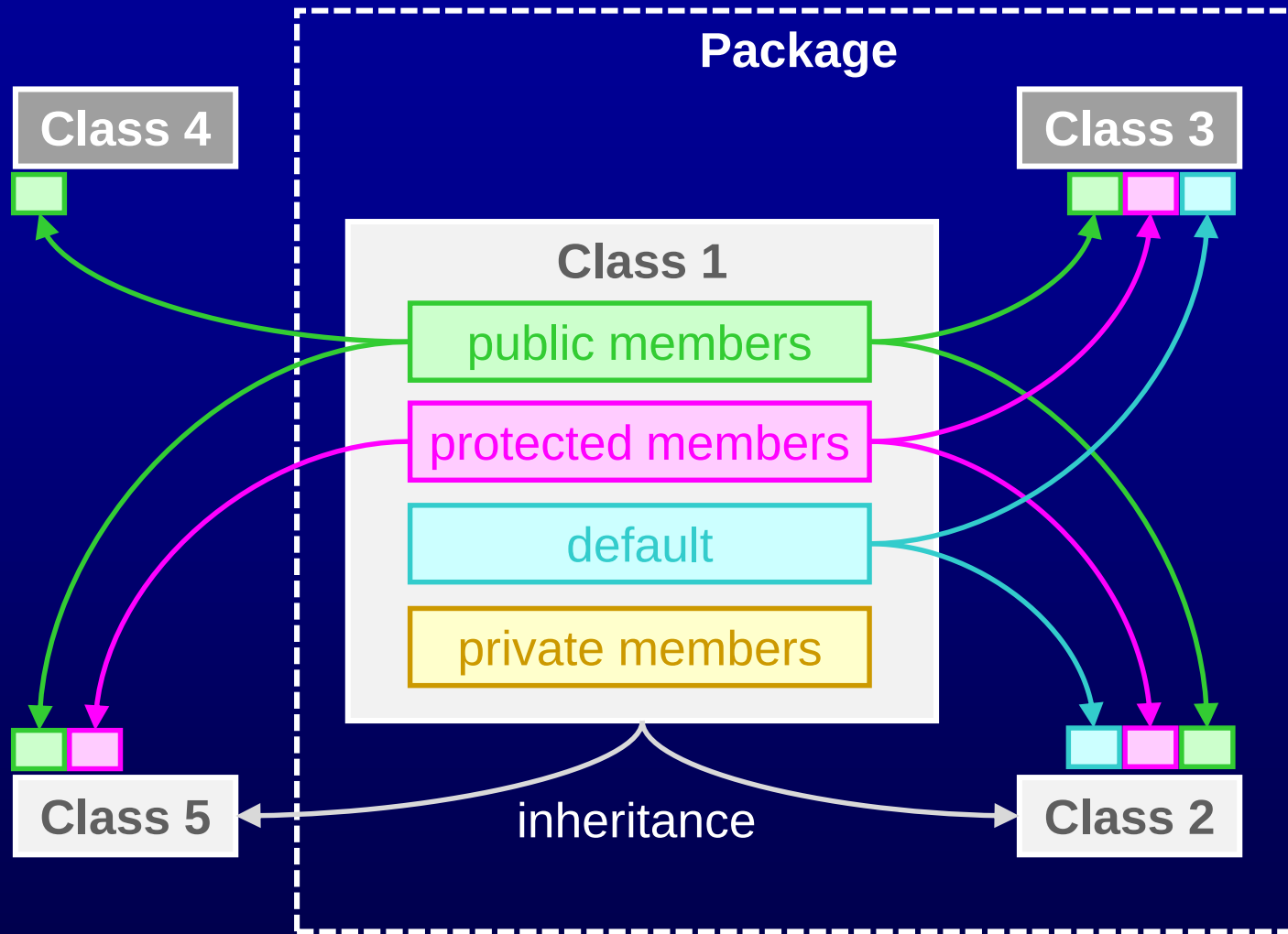
Access Control in Java (2)



Access Control in Java (2)



Access Control in Java (2)



Object-Oriented Language Features (Java)

- **Method overriding**

Allows a method in a subclass to have the same name, arguments and result type as a method in its parent

- **Variable hiding**

Achieved by defining a variable in a child class that has the same name and type of an inherited variable

- **Class constructors**

Not inherited in the same way other methods are – must be explicitly called

- **Each object has ...**

- a *declared* type : ***Parent P;***
- an *actual* type : ***P = new Child ();*** or assignment : ***P = Pold;***
- Declared and actual types allow uses of the same name to reference **different variables** with different **types**

OO Language Feature Terms

- **Polymorphic attribute**
 - An object reference that can take on various types
 - Type the object reference takes on during execution can change
- **Polymorphic method**
 - Can accept parameters of different types because it has a parameter that is declared of type Object
- **Overloading**
 - Using the same name for different constructors or methods in the same class
- **Overriding**
 - A child class declares an object or method with a name that is already declared in an ancestor class
 - Easily confused with overloading because the two mechanisms have similar names and semantics
 - Overloading is in the same class, overriding is between a class and a descendant

More OO Language Feature Terms

- Members associated with a class are called **class** or **instance** variables and methods
 - **Static methods** can operate only on static variables; not instance variables
 - **Instance variables** are declared at the class level and are available to objects
- 20 object-oriented mutation operators **defined for Java** – muJava
- Broken into **4 general categories**

Class Mutation Operators for Java

(1) Encapsulation

AMC

(2) Inheritance

HVD, HVI, OMD, OMM, OMR, SKD, PCD

(3) Polymorphism

ATC, DTC, PTC, RTC, OMC, OMD, AOC, ANC

(4) Java-Specific

TKD, SMC, VID, DCD

OO Mutation Operators—*Encapsulation*

1. *AMC — Access Modifier Change*

The access level for each instance variable and method is changed to other access levels.

OO Mutation Operators—*Example*

1. AMC – Access Modifier Change

point	
	private int x;
Δ1	public int x;
Δ2	protected int x;
Δ3	int x;

OO Mutation Operators—*Inheritance*

2. HVD — *Hiding Variable Deletion*

Each declaration of an overriding or hiding variable is deleted.

3. HVI — *Hiding Variable Insertion*

A declaration is added to hide the declaration of each variable declared in an ancestor.

4. OMD — *Overriding Method Deletion*

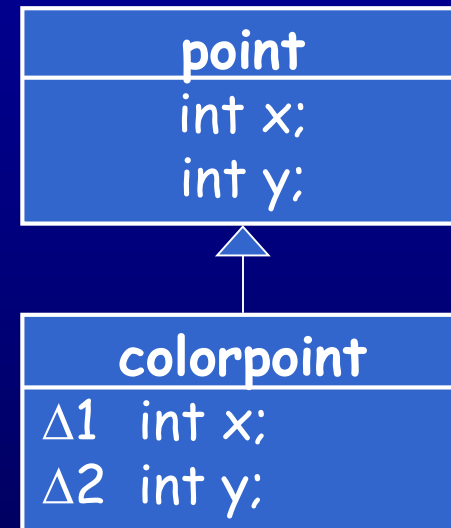
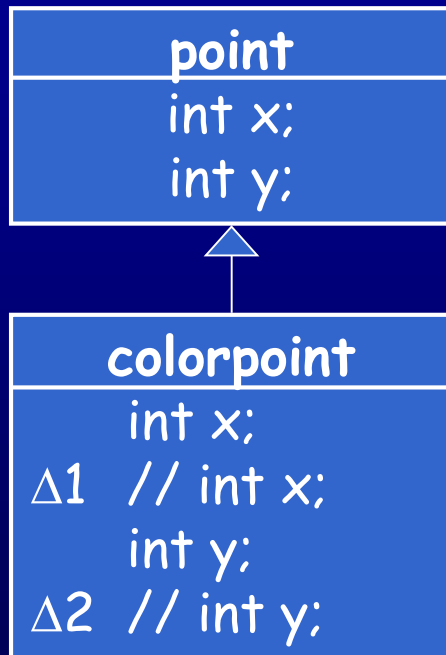
Each entire declaration of an overriding method is deleted.

5. OMM — *Overridden Method Moving*

Each call to an overridden method is moved to the first and last statements of the method and up and down one statement.

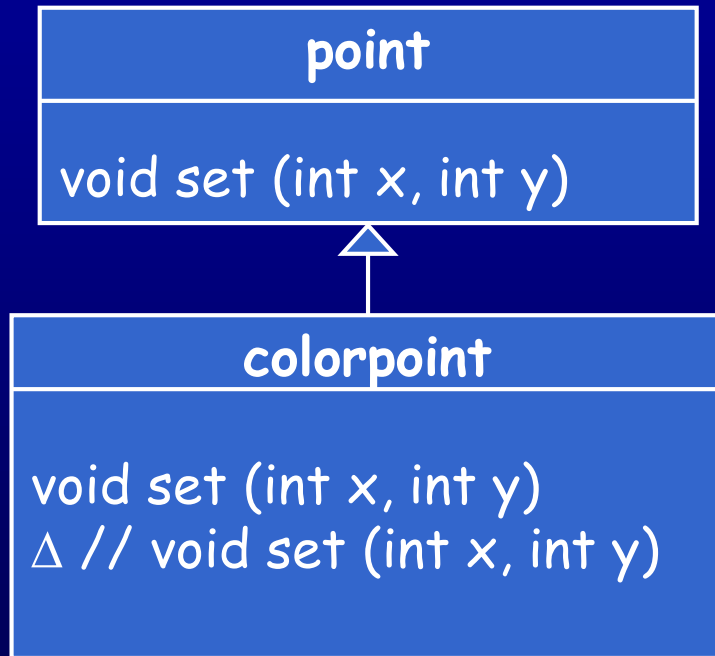
OO Mutation Operators—*Example*

2. *HVD – Hiding Variable Deletion* 3. *HVI – Hiding Variable Insertion*

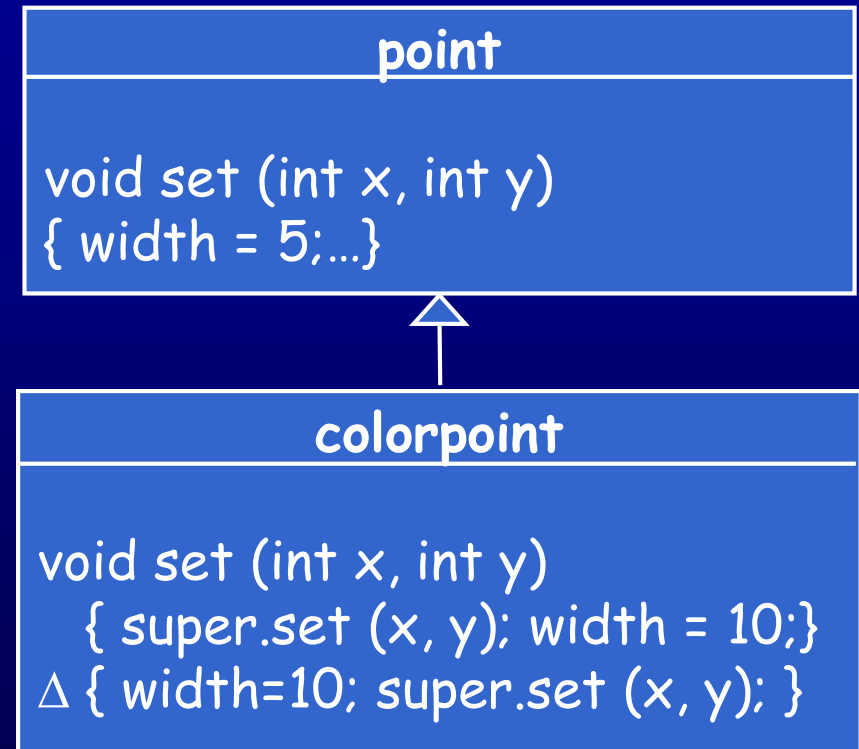


OO Mutation Operators—*Example*

4. OMD – Overriding Method Deletion



5. OMM – Overriding Method Moving



OO Mutation Operators—*Inheritance*

6. OMR — *Overridden Method Rename*

Renames the parent's versions of methods that are overridden in a subclass so that the overriding does not affect the parent's method.

7. SKD — *Super Keyword Deletion*

Delete each occurrence of the **super** keyword.

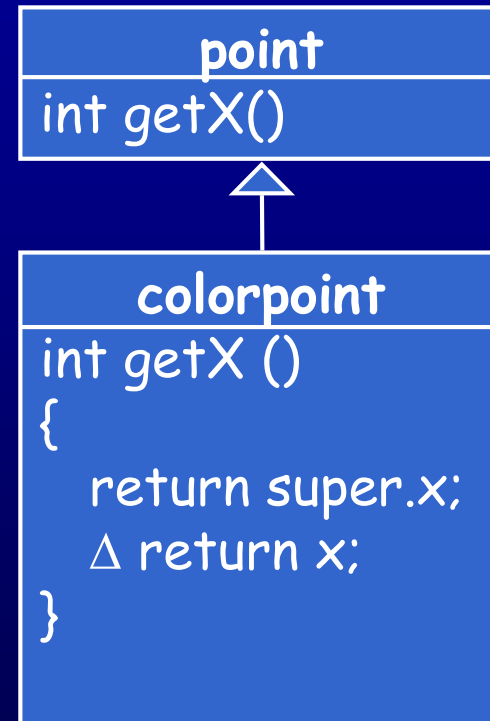
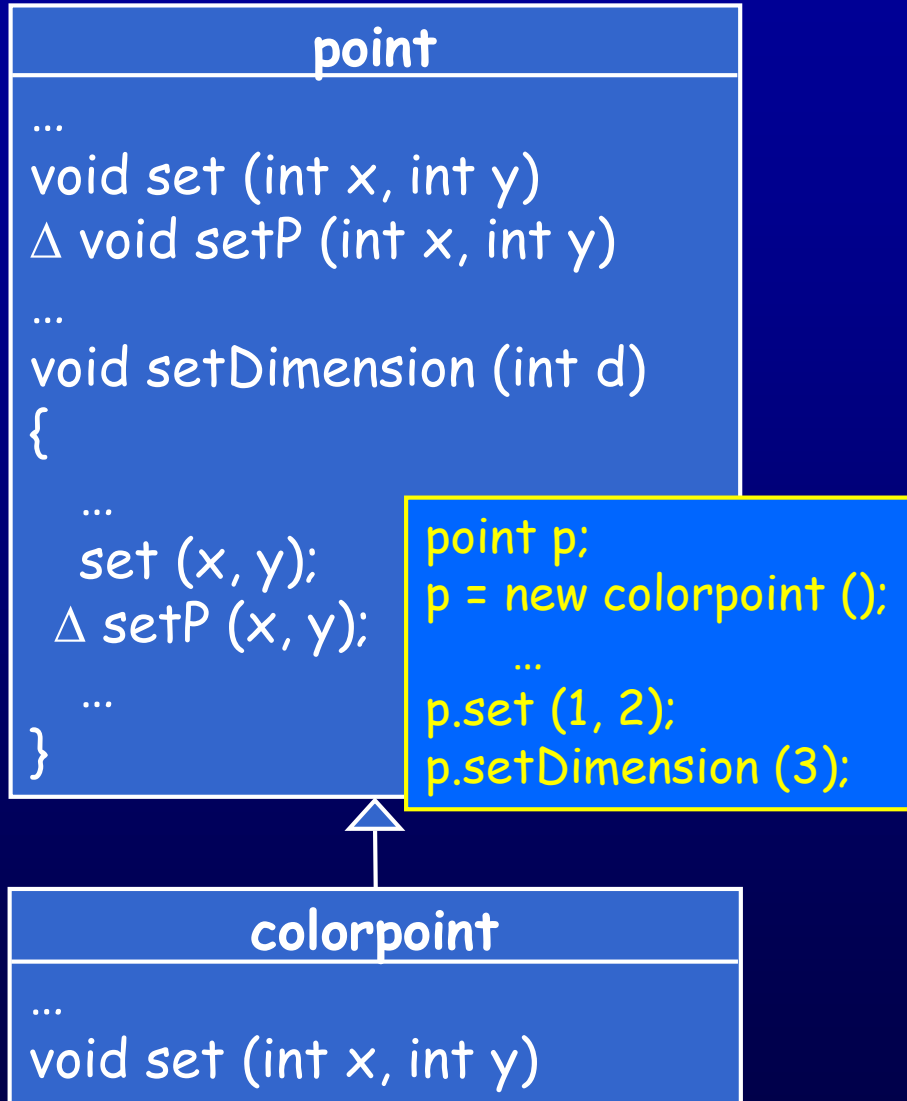
8. PCD — *Parent Constructor Deletion*

Each call to a **super** constructor is deleted.

OO Mutation Operators—*Example*

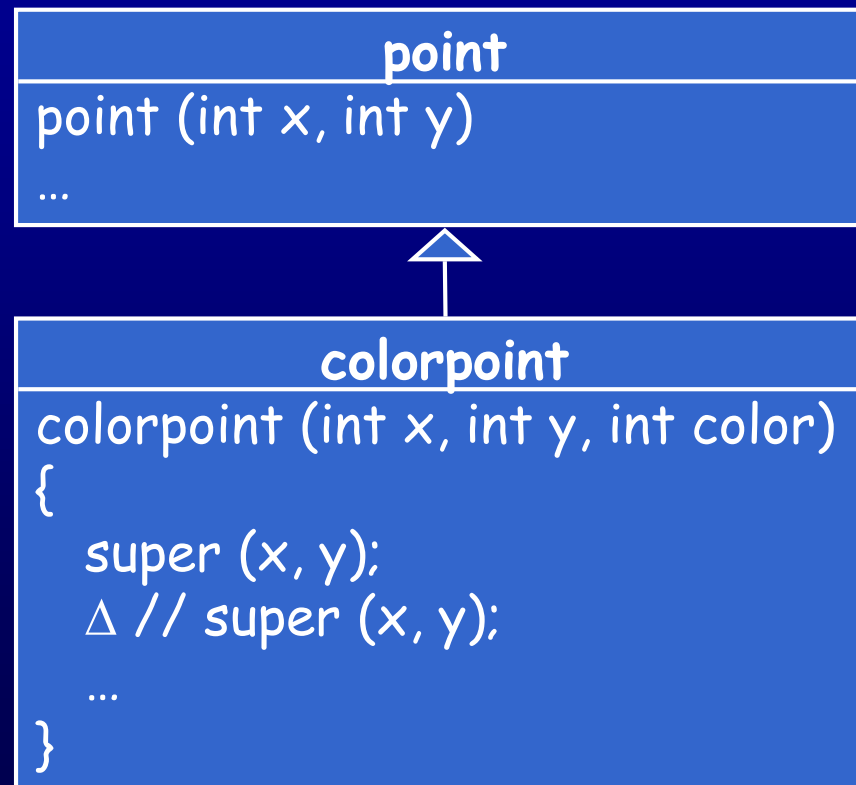
6. OMR – Overriding Method Rename

7. SKD – Super Keyword Deletion



OO Mutation Operators—*Example*

8. PCD – Parent Constructor Deletion



OO Mutation Operators—*Polymorphism*

9. ATC — *Actual Type Change*

The actual type of a new object is changed in the `new()` statement.

10. DTC — *Declared Type Change*

The declared type of each new object is changed in the declaration.

11. PTC — *Parameter Type Change*

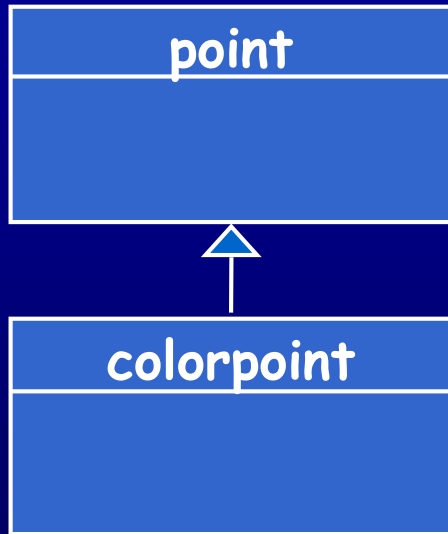
The declared type of each parameter object is changed in the declaration.

12. RTC — *Reference Type Change*

The right side objects of assignment statements are changed to refer to objects of a compatible type.

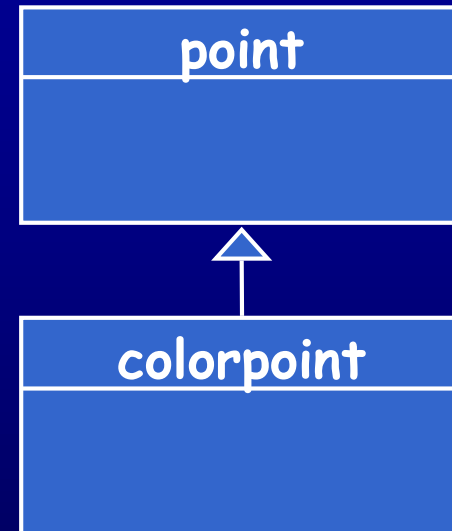
OO Mutation Operators—*Example*

9. ATC – Actual Type Change



```
point p;  
p = new point ();  
Δ p = new colorpoint ();
```

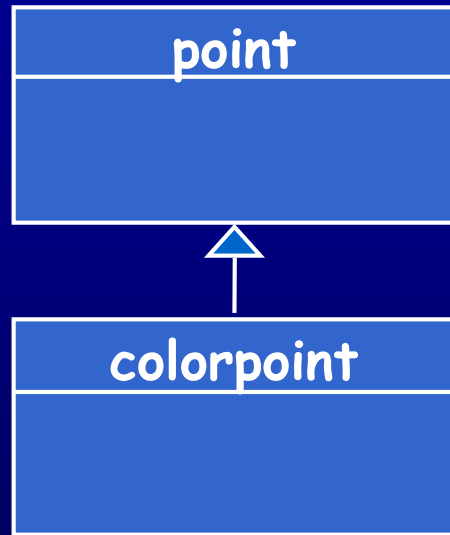
10. DTC – Declared Type Change



```
point p;  
Δ colorpoint p;  
p = new colorpoint ();
```

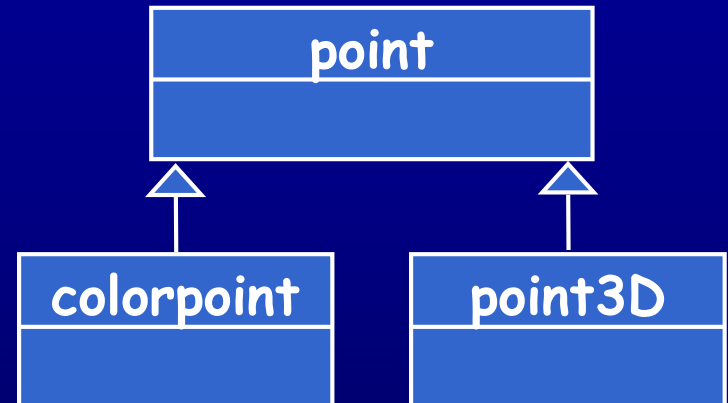
OO Mutation Operators—*Example*

11. PTC – Parameter Type Change



```
boolean equals (point p)
{ ... }
Δ boolean equals (colorpoint p)
{ ... }
```

12. RTC – Reference Type Change



```
point p;
colorpoint cp = new colorpoint (0, 0);
point3D p3d = new point3D (0, 0, 0);
p = cp;
Δ p = p3d;
```

OO Mutation Operators—*Polymorphism*

13. OMC — *Overloading Method Change*

For each pair of methods that have the same name, the bodies are interchanged.

14. OMD — *Overloading Method Deletion*

Each overloaded method declaration is deleted, one at a time.

15. AOC — *Argument Order Change*

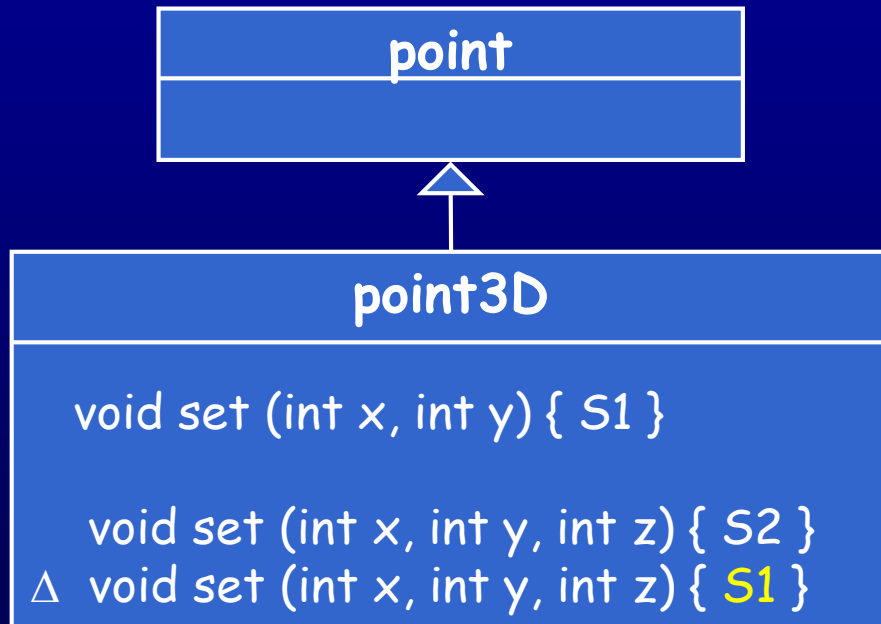
The order of the arguments in method invocations is changed to be the same as that of another overloading method, if one exists.

16. ANC — *Argument Number Change*

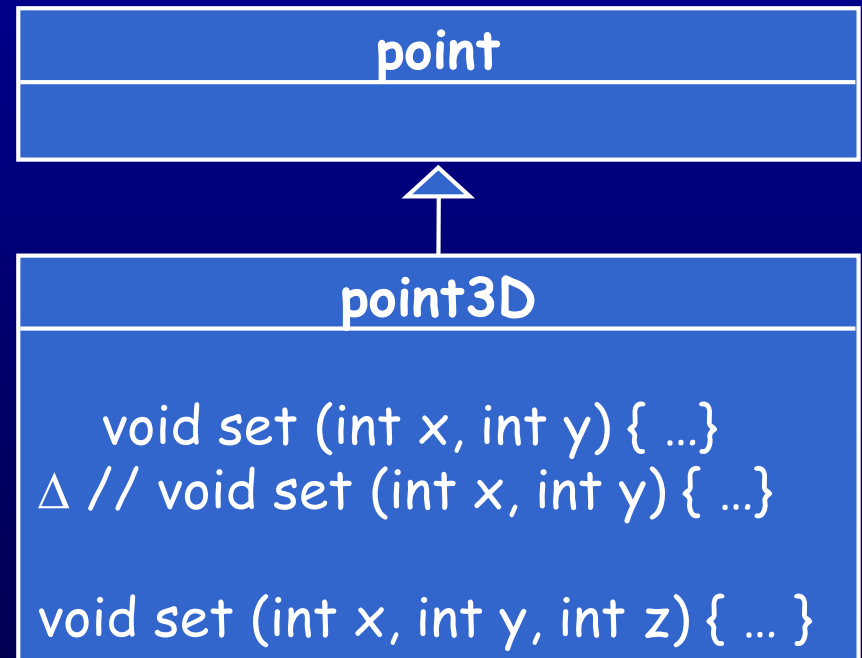
The number of the arguments in method invocations is changed to be the same as that of another overloading method, if one exists.

OO Mutation Operators—*Example*

13. OMC – Overloading Method Change



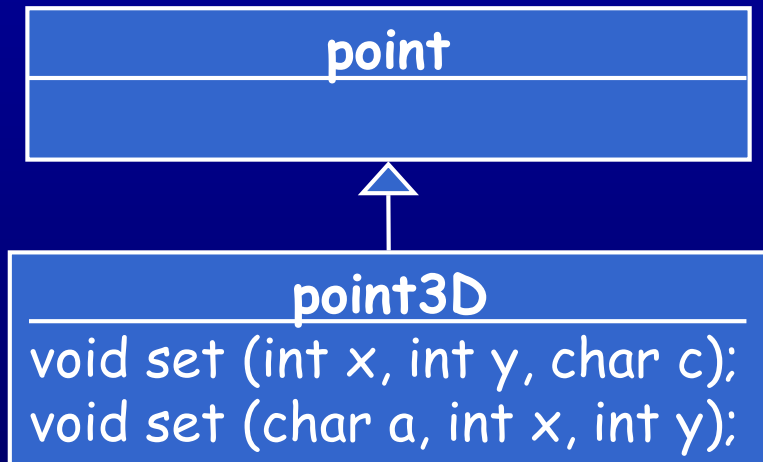
14. OMD – Overloading Method Deletion



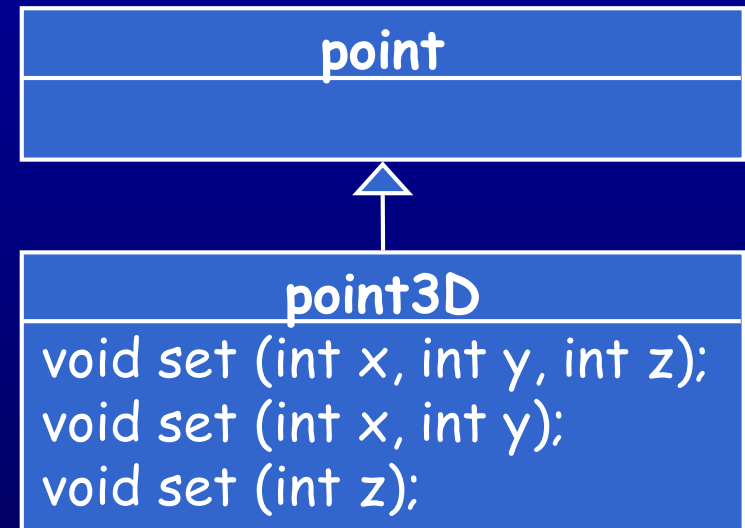
OO Mutation Operators—*Example*

15. AOC – Argument Order Change

16. ANC – Argument Number Change



```
point3D p;
p.set (1, 2, 't');
Δ p.set ('t', 1, 2);
```



```
point3D p;
p.set (1, 2, 3);
Δ p.set (2, 3);
Δ p.set (3);
```

OO Mutation Operators—*Language Specific*

17. TKD — *this Keyword Deletion*

Each occurrence of the keyword **this** is deleted.

18. SMC — *Static Modifier Change*

Each instance of the **static** modifier is removed, and the **static** modifier is added to instance variables.

19. VID — *Variable Initialization Deletion*

Remove initialization of each member variable.

20. DCD — *Default Constructor Delete*

Delete each declaration of default constructor (with no parameters).

OO Mutation Operators—*Example*

17. TKD – This Keyword Deletion

point
...
void set (int x, int y)
{
this.x = x;
Δ1 x = x;
this.y = y;
Δ2 y = y;
}
...

18. SMC – Static Modifier Change

point
public static int x = 0;
Δ1 public int x = 0;
public int y = 0;
Δ2 public static int y = 0;
...

OO Mutation Operators—*Example*

19. VID –Variable Initialization Deletion

point
<pre>int x = 5; Δ int x; ...</pre>

20. DCD – Default Constructor Delete

point
<pre>point() { ... } Δ // point() { ... } ...</pre>

Integration Mutation Summary

- Integration testing often looks at **couplings**
- We have not used **grammar testing** at the integration level
- Mutation testing modifies **callers** and **callees**
- **OO mutation** focuses on inheritance, polymorphism, dynamic binding, information hiding and overloading
 - The access levels make it easy to make mistakes in OO software
- **muJava** is an educational / research tool for mutation testing of Java programs
 - <http://cs.gmu.edu/~offutt/mujava/>

Introduction to Software Testing

Chapter 5.4

Model-Based Grammars

Paul Ammann & Jeff Offutt

<http://www.cs.gmu.edu/~offutt/softwaretest/>

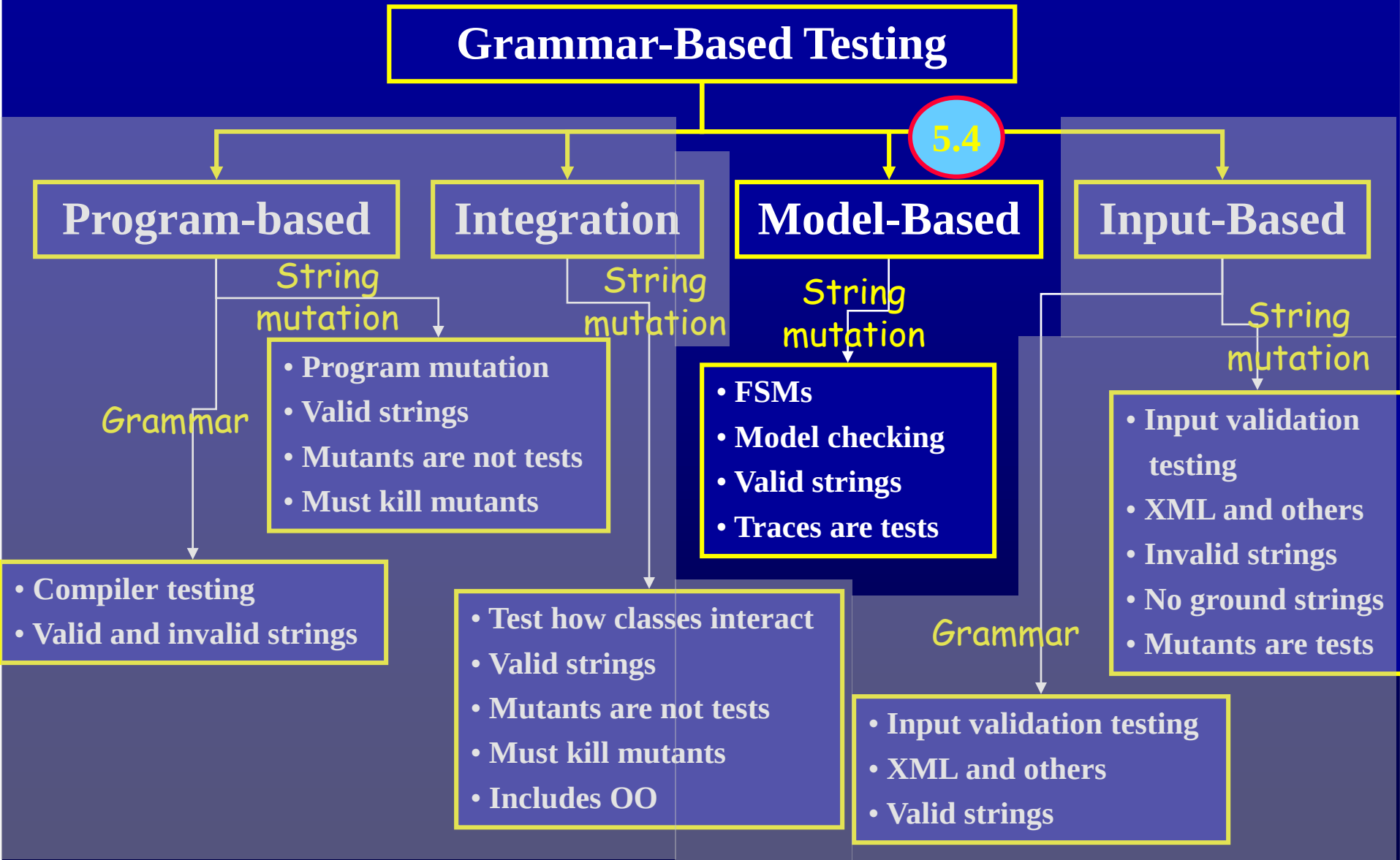
Model-based Grammars

Model-based

Languages that describe software in abstract terms

- **Formal** specification languages
 - Z, SMV, OCL, ...
- **Informal** specification languages
- **Design** notations
 - Statecharts, FSMs, UML notations
- **Model-based** languages are becoming more widely used

Instantiating Grammar-Based Testing



BNF Grammar Testing (5.4.1)

- Terminal symbol coverage and production coverage have only been applied to **algebraic** specifications
- Algebraic specifications are **not widely used**
- This is essentially **research-only**, so not covered in this book

Specification-based Mutation (5.4.2)

- A **finite state machine** is essentially a graph G
 - Nodes are states
 - Edges are transitions
- A *formalization* of an FSM is:
 - States are **implicitly defined** by declaring variables with limited range
 - The *state space* is then the **Cartesian product** of the ranges of the variables
 - *Initial states* are defined by **limiting the ranges** of some or all of the variables
 - *Transitions* are defined by **rules** that characterize the source and target of each transition

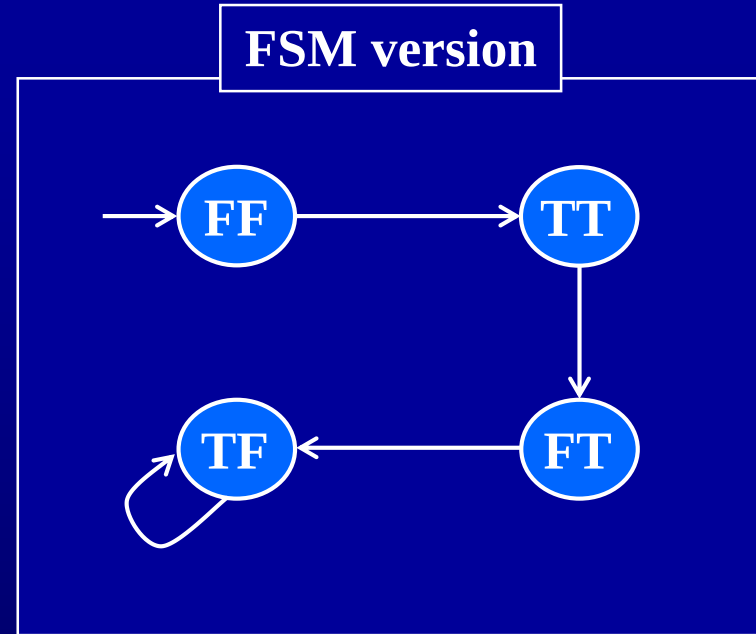
Example SMV Machine

```
MODULE main
#define false 0
#define true 1
VAR
    x, y : boolean;
ASSIGN
    init (x) := false;
    init (y) := false;
    next (x) := case
        !x & y : true;
        !y      : true;
        x       : false;
        true    : x;
    esac;
    next (y) := case
        x & !y : false;
        x & y  : y;
        !x & y : false;
        true  : true;
    esac;
```

- Initial state : (F, F)
- Value for **x** in next state:
 - if **x=F** and **y=T**, next state has **x=T**
 - if **y=F**, next state has **x=T**
 - if **x=T**, next state has **x=F**
 - otherwise, next state **x** does not change
- Value for **y** in next state:
 - if (T, F), next state has **y=F**
 - if (T, T), next state **y** does not change
 - if (F, T), next state has **y=F**
 - otherwise, next state has **y=T**
- Any ambiguity in SMV is resolved by the order of the cases
- “**true : x**” corresponds to “**default**” in programming

Example SMV Machine

```
MODULE main
#define false 0
#define true 1
VAR
    x, y : boolean;
ASSIGN
    init (x) := false;
    init (y) := false;
    next (x) := case
        !x & y : true;
        !y      : true;
        x       : false;
        true    : x;
    esac;
    next (y) := case
        x & !y : false;
        x & y  : y;
        !x & y : false;
        true   : true;
    esac;
```



- Converting from SMV to FSM is mechanical and easy to automate
- SMV notation is **smaller** than graphs for **large** finite state machines

Using SMV Descriptions

- Finite state descriptions can capture **system behavior** at a very high level – suitable for communicating with end users
- The verification community has built **powerful analysis tools** for finite state machines expressed in SMV
- These tools produce **explicit evidence** for properties that are **not true**
- This “*evidence*” is presented as sequences of states, called “**counterexamples**”
- Counterexamples are *paths* through the FSM that can be used as **test cases**

Mutations and Test Cases

- Mutating FSMs requires **mutation operators**
- Most FSM mutation operators are **similar** to program language operators

Constant Replacement operator:

- changes a constant to each other constant
- in the *next(y)* case: $!x \ \& \ y : \text{false}$ is mutated to $!x \ \& \ y : \text{true}$
- To kill this mutant, we need a **sequence of states** (a path) that the **original machine allows** but the mutated machine **does not**
- This is what **model checkers** do
 - Model checkers find **counterexamples** – paths in the machine that violate some **property**
 - Properties are written in “**temporal logic**” – logical statements that are true for some period of time
 - $!x \ \& \ y : \text{false}$ has different result from $!x \ \& \ y : \text{true}$

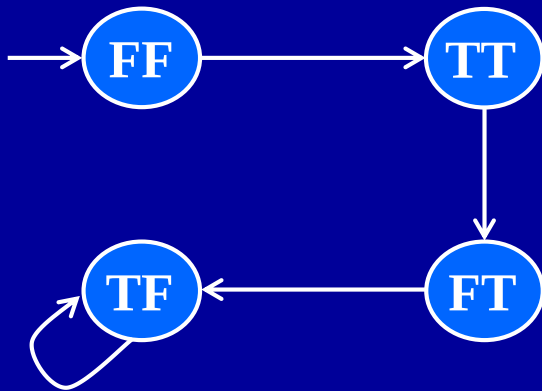
Counter-Example for FSM

```
assume x is input.  
next (y) := case  
    x & !y : false;  
    x & y  : y;  
    !x & y : false;  
Δ 1    !x & y : true;  
    true  : true;  
esac;
```

written in
SMV as

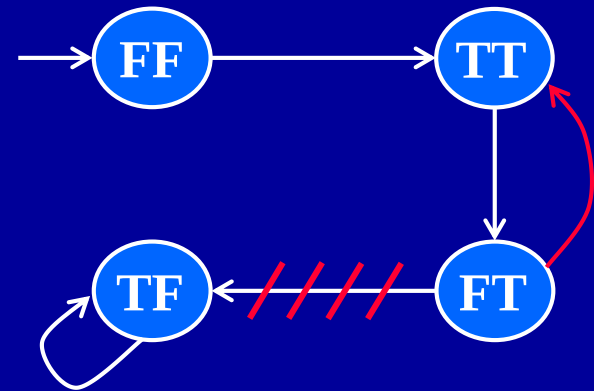
```
assume x is input.  
next (y) := case  
    x & !y : false;  
    x & y  : y;  
    !x & y : false;  
    true  : true;  
esac;  
SPEC AG (!x & y) → AX (y=true)
```

FSM version



mutated
FSM

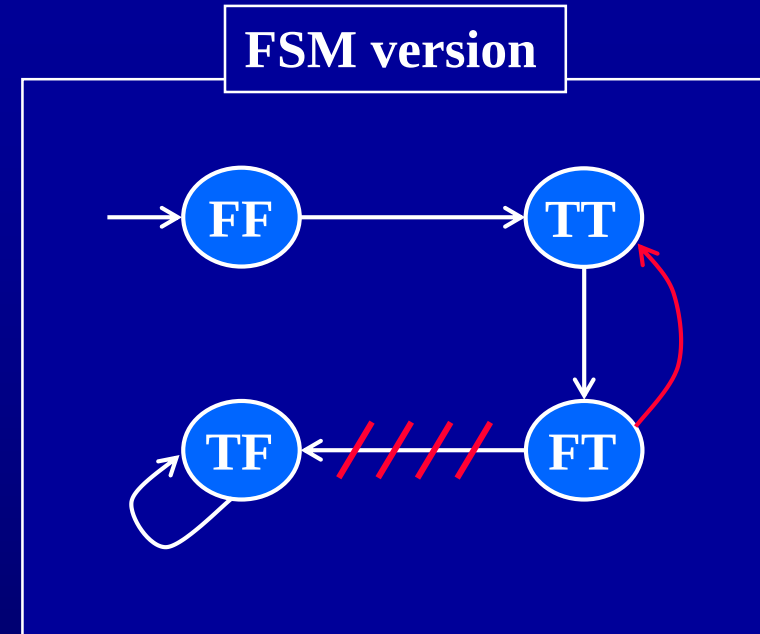
FSM version



Counter-Example for FSM

- The model checker should produce:

```
/* state 1 */ { x = 0, y = 0 }  
/* state 2 */ { x = 1, y = 1 }  
/* state 3 */ { x = 0, y = 1 }  
/* state 4 */ { x = 1, y = 0 }
```



- This represents a test case that goes from nodes FF to TT to FT to TF in the original FSM
 - The last step in the mutated FSM will be to TT, killing the mutant
- If no sequence is produced, the mutant is equivalent
 - Equivalence is *undecidable* for programs, but *decidable* for FSMs

Model-Based Grammars Summary

- **Model-checking** is slowly growing in use
- **Finite state machines** can be encoded into model checkers
- Properties can be defined on FSMs and model checking used to find **paths that violate** the properties
- **No equivalent** mutants
- Everything is **finite**

Introduction to Software Testing

Chapter 5.5

Input Space Grammars

Paul Ammann & Jeff Offutt

<http://www.cs.gmu.edu/~offutt/softwaretest/>

Input Space Grammars

Input Space

The set of allowable inputs to software

- The input space can be **described** in many ways
 - User manuals
 - Unix man pages
 - Method signature / Collection of method preconditions
 - A language
- Most input spaces can be described as **grammars**
- Grammars are usually not provided, but **creating them** is a valuable service by the tester
 - Errors will often be found simply by creating the grammar

Using Input Grammars

- Software should **reject** or **handle** invalid data
- Programs often do this **incorrectly**
- Some programs (rashly) **assume** all input data is correct
- Even if it works **today** ...
 - What about after the program goes through some **maintenance changes** ?
 - What about if the component is **reused** in a new program ?
- Consequences can be **severe** ...
 - The **database** can be corrupted
 - **Users** are not satisfied
 - Many **security vulnerabilities** are due to unhandled exceptions ... from invalid data

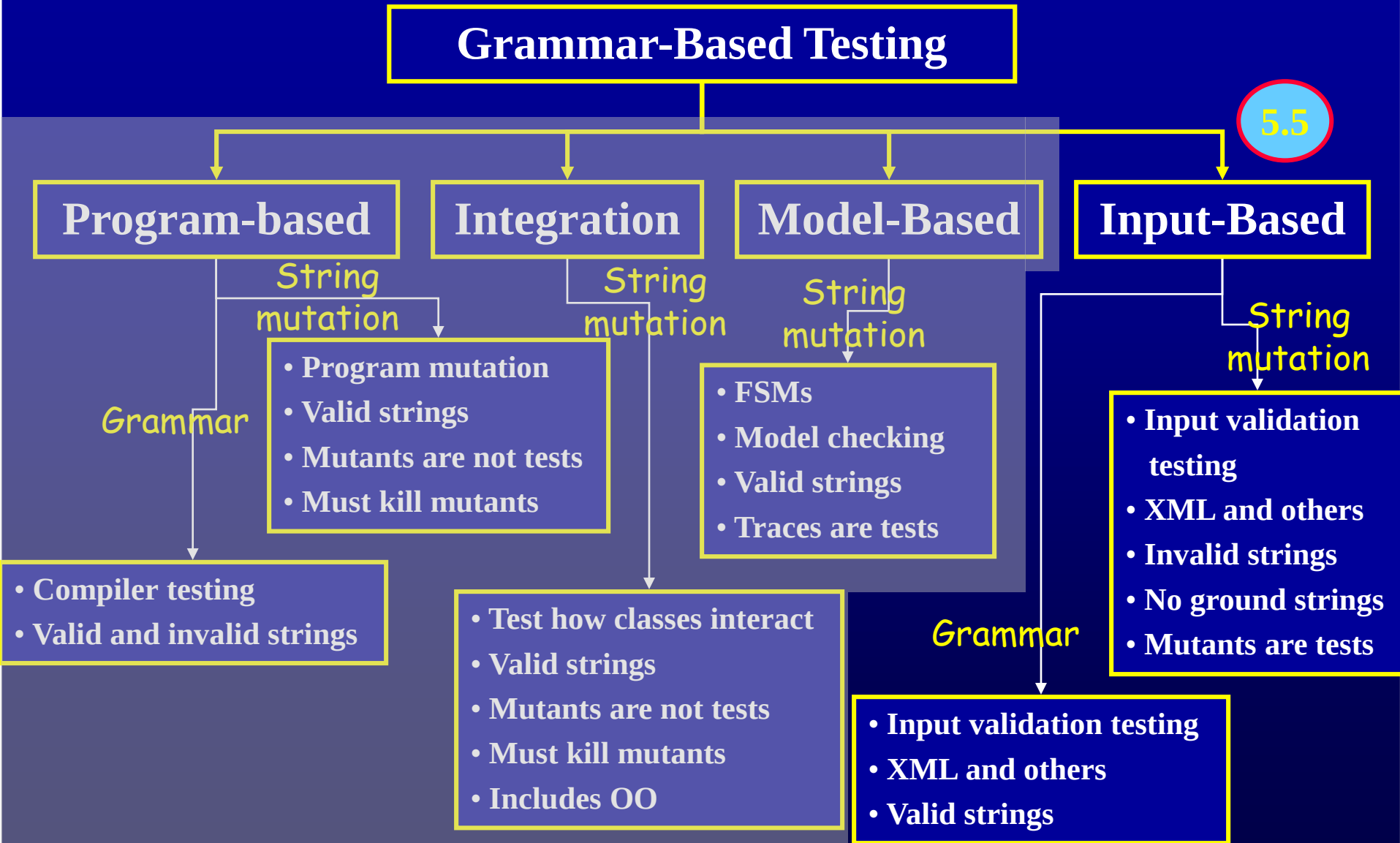
Validating Inputs

Input Validation

Deciding if input values can be processed by the software

- Before starting to process inputs, wisely written programs check that the **inputs are valid**
- How should a program **recognize** invalid inputs ?
- What should a program **do with** invalid inputs ?
- If the input space is described as a grammar, a **parser** can check for validity automatically
 - This is very **rare**
 - It is easy to write input checkers – but also easy to make mistakes

Instantiating Grammar-Based Testing



Input Space BNF Grammars (5.5.1)

- Input spaces can be expressed in many forms
- A common way is to use some form of **grammar**
- We will look at **three** grammar-based ways to describe input spaces
 1. Regular expressions
 2. BNF grammars
 3. XML and Schema
- All are **similar** and can be used in different contexts

Regular Expressions

Consider a program that processes a sequence of deposits and debits to a bank

Inputs

deposit 5306 \$4.30

debit 0343 \$4.14

deposit 5306 \$7.29

Initial Regular Expression

(deposit account amount | debit account amount) *



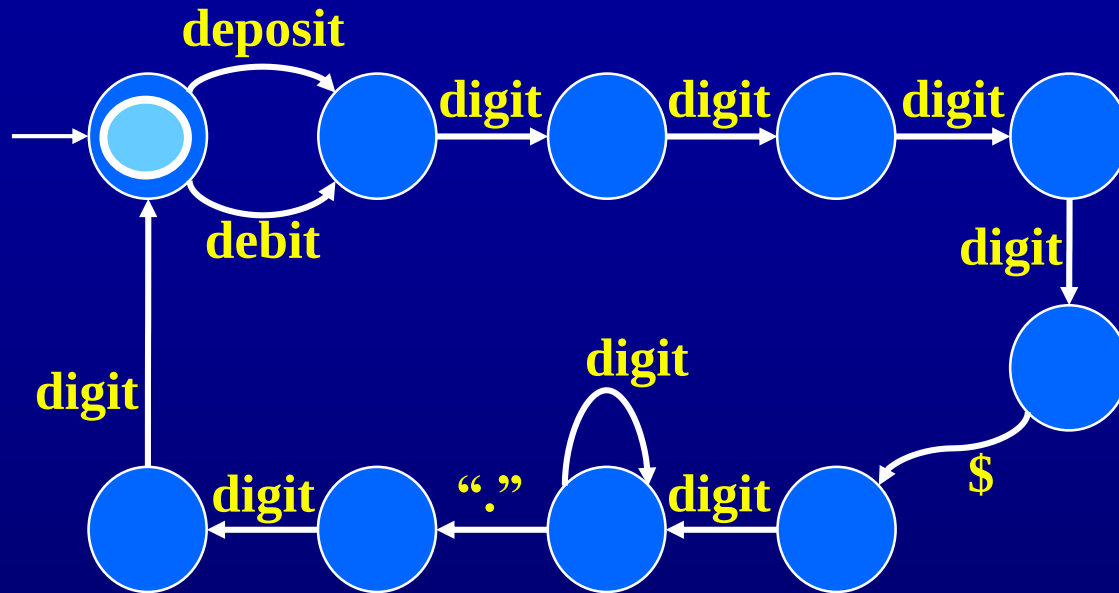
FSM to represent the grammar

BNF Grammar for Bank Example

Grammars are more expressive than regular expressions—they can capture more details

```
bank    ::= action*
action  ::= dep | deb
dep     ::= "deposit" account amount
deb     ::= "debit" account amount
account ::= digit4
amount  ::= "$" digit+ "." digit2
digit   ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" |
           "7" | "8" | "9"
```

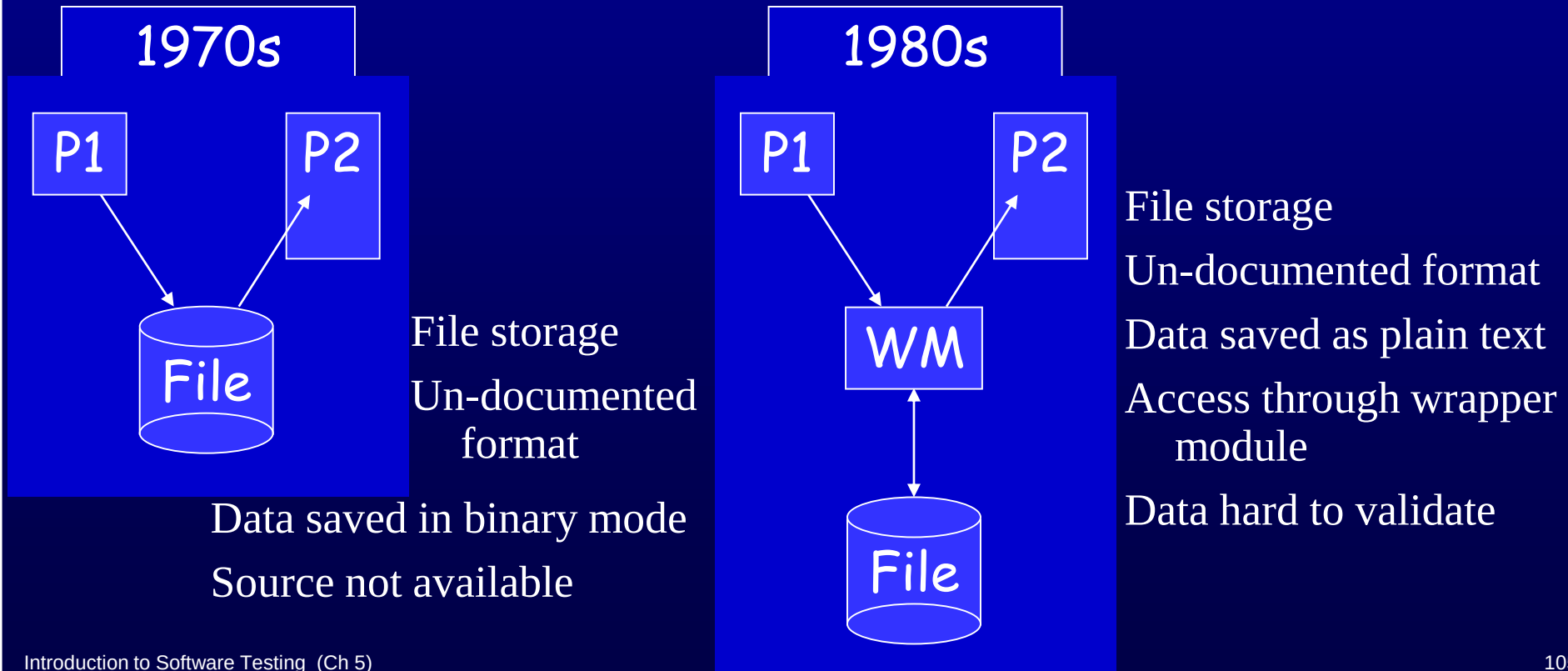
FSM for Bank Grammar



- Derive tests by **systematically replacing** each non-terminal with a production
- If the tester designs the grammar from informal input descriptions, **do it early**
 - In time to **improve** the design
 - **Mistakes** and **omissions** will almost always be found

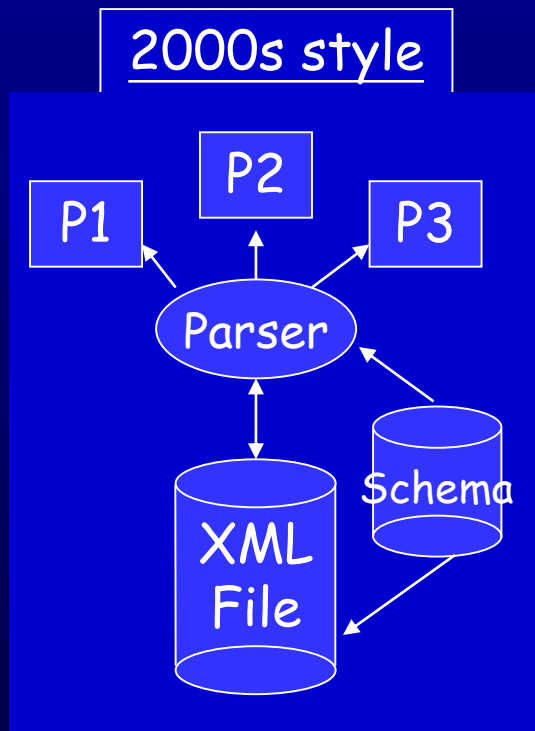
Using XML to Describe Input Spaces

- Software components that pass data must agree on **format**, **types**, and **organization**
- Web applications have **unique requirements** :
 - Very **loose coupling** and **dynamic integration**



XML in Very Loosely Coupled Software

- Data is passed **directly** between components
- XML allows data to be **self-documenting**



- P1, P2, and P3 can see the format, contents, and structure of the data
- Data sharing is independent of type
- Format is easy to understand
- Grammars are defined in DTDs or Schemas

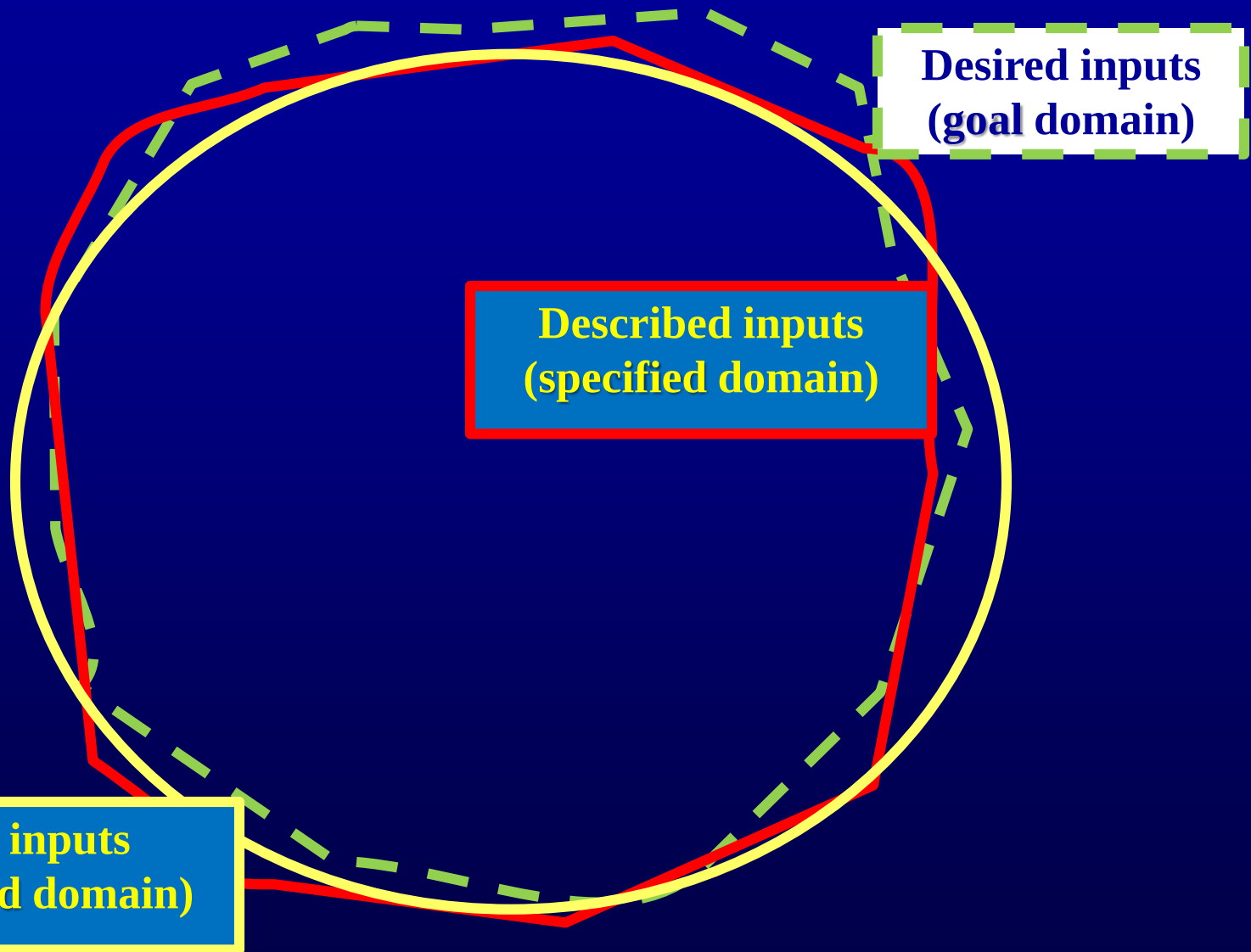
document type definition

XML for Book Example

```
<books>  
  <book>  
    <ISBN>0471043281</ISBN>  
    <title>The Art of Software Testing</title>  
    <author>Glen Myers</author>  
    <publisher>Wiley</publisher>  
    <price>50.00</price>  
    <year>1979</year>  
  </book>  
</books>
```

- XML messages are defined by **grammars**
 - **Schemas** and DTDs
 - Schemas can define many kinds of **types**
 - Schemas include “**facets**,” which refine the grammar
- schemas define input spaces for software components

Representing Input Domains



Example Input Domains

- Goal domains are often **irregular**
- **Goal** domain for **credit cards**[†]
 - First digit is the Major Industry Identifier
 - First 6 digits and length specify the issuer
 - Final digit is a “check digit”
 - Other digits identify a specific account
- Common **specified** domain
 - First digit is in { 3, 4, 5, 6 } (travel and banking)
 - Length is between 13 and 16
- Common **implemented** domain
 - All digits are numeric

[†] More details are on : <http://www.merriampark.com/anatomycc.htm>

Representing Input Domains

*This region is
a rich source
of software
errors ...*

goal domain

specified domain

*This region is a
rich source of
software errors ...*

implemented domain

*This region is
a rich source
of software
errors ...*

Using Grammars to Design Tests

- This form of testing allows us to focus on **interactions** among the components
 - Originally applied to Web services, which depend on XML
- A **formal model** of the XML grammar is used
- The grammar is used to create **valid** as well as **invalid** tests
- The grammar is **mutated**
- The mutated grammar is used to generate new **XML messages**
- The XML messages are used as **test cases**

Book Grammar – Schema

```
<xs:element name = “books”>
  <xs:complexType>
    <xs:sequence>
      <xs:element name = “book” maxOccurs = “unbounded”>
        <xs:complexType>
          <xs:sequence>
            <xs:element name = “ISBN” type = “isbnType” minOccurs = “0” />
            <xs:element name = “author” type = “xs:string” />
            <xs:element name = “title” type = “xs:string” />
            <xs:element name = “publisher” type = “xs:string” />
            <xs:element name = “price” type = “priceType” />
            <xs:element name = “year” type = “yearType” />
          </xs:sequence>
        </xs:complexType>
      </xs:element>
    </xs:sequence>
  </xs:complexType>
</xs:element>
```

Built-in types

priceType

```
<xs:simpleType name = “priceType”>
  <xs:restriction base = “xs:decimal”>
    <xs:fractionDigits value = “2” />
    <xs:maxInclusive value = “1000.00” />
  </xs:restriction>
</xs:simpleType>
```

XML Constraints – “Facets”

Boundary Constraints	Non-boundary Constraints
maxOccurs	enumeration
minOccurs	use
length	fractionDigits
maxExclusive	pattern
maxInclusive	nillable
maxLength	whiteSpace
minExclusive	unique
minInclusive	
minLength	
totalDigits	

Generating Tests

- **Valid** tests

- Generate tests as **XML messages** by deriving strings from grammar
- Take **every production** at least once
- Take **choices** ... “maxOccurs = “unbounded” means use 0, 1 and more than 1

- **Invalid** tests

- **Mutate** the grammar in structured ways
- Create XML messages that are “**almost**” valid
- This explores the **gray space** on the previous slide

Generating Tests

- The criteria in section 5.1.1 can be used to generate tests
 - **Production** and **terminal symbol** coverage
- The only choice in books is based on “**minOccurs**”
- PC requires **two tests**
 - ISBN is present
 - ISBN is **not** present
- The facets are used to generate values that are valid
 - We also want values that are **not** valid ...

Mutation for Input Grammars (5.5.2)

- Software should **reject** or **handle** invalid data
- A very **common mistake** is for programs to do this incorrectly
- Some programs (rashly) **assume** that all input data is correct
- Even if it works today ...
 - What about after the program goes through some **maintenance changes** ?
 - What about if the component is **reused** in a new program ?
- Consequences can be **severe** ...
 - Most **security vulnerabilities** are due to unhandled exceptions ... from invalid data
- To test for invalid data (including security testing), **mutate the grammar**

Mutating Input Grammars

- Mutants are **tests**
- Create **valid** and **invalid** strings
- No **ground strings** – no killing
 - mutation to production rules, instead of real programs, interfaces, ...
- Mutation operators listed here are **general** and should be refined for specific grammars

Input Grammar Mutation Operators

1. *Nonterminal Replacement*

Every nonterminal symbol in a production is replaced by other nonterminal symbols.

2. *Terminal Replacement*

Every terminal symbol in a production is replaced by other terminal symbols.

3. *Terminal and Nonterminal Deletion*

Every terminal and nonterminal symbol in a production is deleted.

4. *Terminal and Nonterminal Duplication*

Every terminal and nonterminal symbol in a production is duplicated.

Mutation Operators

- Many strings may **not be useful**
- Use additional **type information**, if possible
- Use **judgment** to throw tests out
- Only apply replacements if “**they make sense**”
- **Examples ...**

Nonterminal Replacement

dep ::= “deposit” account amount
dep ::= “deposit” amount amount
dep ::= “deposit” account digit

deposit \$1500.00 \$3789.88
deposit 4400 5

Terminal Replacement

amount ::= “\$” digit⁺ “.” digit²
amount ::= “.” digit⁺ “.” digit²
amount ::= “\$” digit⁺ “\$” digit²
amount ::= “\$” digit⁺ “1” digit²

deposit 4400 .1500.00
deposit 4400 \$1500\$00
deposit 4400 \$1500100

Terminal and Nonterminal Deletion

dep ::= “deposit” account amount
dep ::= account amount
dep ::= “deposit” amount
dep ::= “deposit” account

4400 \$1500.00
deposit \$1500.00
deposit 4400

Terminal and Nonterminal Duplication

dep ::= “deposit” account amount
dep ::= “deposit” “deposit” account amount
dep ::= “deposit” account account amount
dep ::= “deposit” account amount amount

deposit deposit 4400 \$1500.00
deposit 4400 4400 \$1500.00
deposit 4400 \$1500.00 \$1500.00

Notes and Applications

- We have more **experience** with program-based mutation than input grammar based mutation
 - Operators are less “**definitive**”
- **Applying** mutation operators
 - **Mutate grammar**, then derive strings
 - Derive strings, **mutate a derivation** “in-process”
- Some mutants give strings in the original grammar (**equivalent**)
 - These strings can **easily be recognized** to be equivalent

Mutating XML

- XML **schemas** can be mutated
- If a schema does not exist, testers should **derive** one
 - As usual, this will help find problems immediately
- Many programs **validate messages** against a grammar
 - Software may still behave correctly, but testers must verify
- Programs are less likely to check all schema **facets**
 - Mutating facets can lead to very effective tests

Test Case Generation – Example

Original Schema (Partial)

```
<xs:simpleType name = “priceType”>  
  <xs:restriction base = “xs:decimal”>  
    <xs:fractionDigits value = “2” />  
    <xs:maxInclusive value = “1000.00” />  
  </xs:restriction>  
</xs:simpleType>
```

Mutants : value = “3”
value = “1”

Mutants : value = “100”
value = “2000”

XML from Original Schema

```
<books>  
  <book>  
    <ISBN>0-201-74095-8</ISBN>  
    <price>37.95</price>  
    <year>2002</year>  
  </book>  
</books>
```

Mutant XML 1
Mutant XML 2
Mutant XML 3
Mutant XML 4

```
<books>  
  <book>  
    <ISBN>0-201-74095-8</ISBN>  
    <price>1500.00</price>  
    <year>2002</year>  
  </book>  
</books>
```

Input Space Grammars Summary

- This application of mutation is **fairly new**
- Automated **tools** do not exist
- Can be used **by hand** in an “ad-hoc” manner to get effective tests
- Applications to **special-purpose grammars** very promising
 - XML
 - SQL
 - HTML